Technische Universität Hamburg

Data Science Foundations

Low-dimensional Control under Embodiment Constraints: From LQR to Reinforcement Learning

Bachelor's Thesis
am Institut Data Science Foundations
Technische Universität Hamburg

vorgelegt von
Alexander Michael Kranz
am
31.10.2026

Erstprüfer:	Prof. Dr. N. Ay
Zweitprüfer	Dr. Manfred Eppe
Betreuung:	Dr. Adwait Datar

Alexander Michael Kranz
geboren am: 09.02.2004
Matrikelnummer: 590156
Studiengang: Data Science

Abstract

Your English abstract here (mandatory if written in English and recommended otherwise).

Zusammenfassung

Hier die deutsche Zusammenfassung einfügen (notwendig).

Contents

1 Introduction **1**

2 Background **3**
2.1 Preliminaries . . . 3
2.1.1 Linear State-Space Models . . . 3
2.1.2 The Double Integrator . . . 5
2.1.3 Characteristics of a System . . . 6
2.1.4 The Linear Quadratic Regulator . . . 10
2.2 Motivation . . . 12
2.3 Approach . . . 14
2.3.1 Uncertainty and Disturbance Models . . . 14
2.3.2 Metrics and Robustness . . . 16

3 Robustness of LQR Solvers in Different Dimensions **23**
3.1 A Known Model . . . 23
3.1.1 Setting . . . 23
3.1.2 Results . . . 25
3.1.3 Noise Experiments . . . 25
3.1.4 Discussion . . . 29
3.2 Unknown Model across Dimensions . . . 29
3.2.1 Setting . . . 30
3.2.2 Results . . . 33
3.2.3 Noise Experiments on Large Inflation . . . 33
3.2.4 Results . . . 33
3.2.5 Discussion . . . 33
3.3 Discussion . . . 33

4 Robustness Of Reinforcement Learning Approaches **35**
4.1 PPO and its Variants . . . 35
4.2 Comparison Across Dimensions . . . 35
4.2.1 Setting . . . 35
4.2.2 Results . . . 35
4.3 Noise On Large Inflation . . . 35
4.3.1 Setting . . . 35

4.3.2 Results . . . 35
4.4 Discussion . . . 35

5 Perception-based Experiments **37**
5.1 Artificial Inflation to Pixel Size . . . 37
5.1.1 Setting . . . 37
5.1.2 Results . . . 37
5.2 The New Environment and Pos-Vel . . . 37
5.3 Effect Of Visual Inputs On The Solvers . . . 37
5.3.1 Setting . . . 37
5.3.2 Results . . . 37
5.4 Discussion . . . 37

6 Conclusion **39**

A Nomenclature **41**
A.1 Notation . . . 41
A.2 Noise intensities . . . 43

B Additional Proofs **45**

C Complete Simulation Results **47**
C.1 Learning Rate of Model-based PG . . . 47
C.2 Nuclear-Norm Weight of S-PI . . . 47
C.3 N4SID Identification Length . . . 47
C.4 Sample Size . . . 48
C.5 Learning Rate of Model-free NPG . . . 48
C.6 Step Size and Nuclear-Norm Weight of S-PI (Model-Free) . . . 52

Bibliography **55**

List of Figures

3.1 TODO 26
3.2 TODO 26
3.3 TODO 27
3.4 TODO 28
3.5 TODO 28
3.6 TODO 29
3.7 TODO 29
3.8 TODO 30

C.1 Training suboptimality of Model-based PG over the solver iterations for each learning rate under *large* inflation. Median and range over ten seeds. 48
C.2 Training suboptimality of S-PI over the solver iterations for each nuclear-norm weight λ under *large* inflation. Median and range over ten seeds. 49
C.3 Median training suboptimality of N4SID + Model-based PG per trajectory length T and Hankel horizon under *large* inflation. 49
C.4 Median training suboptimality of N4SID + Model-based PG per trajectory length T and Hankel horizon under *medium* inflation. . . 50
C.5 Training suboptimality of Model-free NPG over the environment steps for each rollout length l and number of trajectories m under *large* inflation. Median and range over ten seeds. 51
C.6 Final training suboptimality of Model-free NPG for each rollout length l and number of trajectories m under *large* inflation. 51
C.7 Training suboptimality of Model-free NPG over the environment steps for each learning rate under *large* inflation. Median and range over ten seeds. 52

List of Tables

2.1 Preset inflation sizes. 20

3.1 Policy architectures optimized by the exact gradients on the true model under *base* inflation, without noise. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 27

3.2 Policy architectures optimized by the exact gradients on the true model under *large* inflation, without noise. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 27

3.3 Policy architectures on the true model, across every inflation size. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 28

3.4 Identification settings per inflation size. 31

A.2 Standard deviation per noise channel and intensity, calibrated to the cost ratio ρ_ℓ of (2.38). Every value reaches its ratio within 0.9 %. 43

C.1 Model-based PG under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 47

C.2 S-PI under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 48

C.3 N4SID + Model-based PG on a system with *medium* inflation with a Hankel horizon of 5. Mean ± standard deviation over 10 runs. . . 48

C.4 N4SID + Model-based PG on a system with *large* inflation with a Hankel horizon of 5. Mean ± standard deviation over 10 runs. Identified counts the seeds a model was obtained from and stable those whose controller left the plant stable. 50

C.5 Model-free NPG: training suboptimality $(J - J^*)/J^*$ per inflation size (rows) and sample-size combination (columns). Mean ± standard deviation over 10 runs. 50

C.6 Model-free NPG under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs. 52
C.7 S-PI (model-free) with $\lambda = 4000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs. 53
C.8 S-PI (model-free) with $\lambda = 40000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs. 53
C.9 S-PI (model-free) with $\lambda = 400000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs. Stable counts the seeds whose controller left the plant stable. 53

Chapter 1

Introduction

Over the recent years, the field of robotics has seen rapid advancements in the capabilities of neural-network-based implementations. An increase in global compute power[CITE: TODO compute], fueled by the so-called "AI race" of LLMs, has allowed researchers around the world to create even more extensive model architectures for more complex use cases [CITE: TODO llm example]. Transferring these structuresto physical bodies requires more space and on-device compute, often the limiting factors of embodiment. To bridge this gap between powerful models and limited resources, the approach of cheap design has become more common. As mentioned by N. Ay[CITE: TODO ay geometric], the core idea is to maintain an "embodied universal approximator", while reducing the overall complexity of the model, which can be defined based on the specific case. The most straightforward approach is inducing the underlying physical dynamics of the embodiment into the architecture, which is often of lower dimensionality than the actual sensors and actuators. Recent papers focusing on algorithms for the LQR[CITE: TODO lqr], apply this approach by offering implementations for enforcing low-rank controllers, leading to lower complexity in relation to parameterization. Still, this low-dimensional approach has yet to be extended[CITE: TODO extension] to more complex embodiments, and is missing a crucial part required for implementation: Robustness. This leaves us with the question: *Is it possible to train a low-dimensional controller for an embodied agent, that is performant yet robust?*

As robustness is a wide topic, in our case, we use it when referring to sensor- and actuator noise, as well as parametric uncertainties of the underlying model and external disturbances. When talking about performance, we mean reducing cost as much as possible in relation to an optimal controller. In the end, we want to show in which cases low-dimensional controllers are a viable alternative, and can thus reduce costs for hardware implementations.

To answer our question, we will first explain the core concepts and collect existing approaches for the LQR setting in Chapter 2. In Chapter 3, we will compare these implementations using the double integrator problem for LQR with respect to gen-

eral performance, as well as robustness. Building on that, we will artificially inflate our inputs and outputs via matrix multiplication in Chapter 4 and additionally introduce deep RL baseline models into the comparison. For Chapter 5, we will completely move away from LQR, and extend the setting to a perception-based implementation. Here we will compare different models with varying parameter complexity in regard to robustness, applying the concepts we learned from our earlier experiments. Finally, we will conclude by analyzing whether we indeed achieved our goal of constructing a robust controller, and stating how our approach can be applied to more complex settings.

Chapter 2

Background

2.1 Preliminaries

Before turning to the experiments, this chapter revisits the concepts that all later parts of this thesis build on. Section 2.1.1 introduces linear state-space models and their discretization, Section 2.1.2 shows them on the double integrator that serves as our running example throughout, and Sections 2.1.3 and 2.1.4 recall the stability notions and the linear quadratic regulator. Sections 2.3.1 and 2.3.2 define the perturbation models and the robustness metrics used to evaluate every controller we construct.

2.1.1 Linear State-Space Models

In control theory, the evolution of an agent together with its environment is described by a *state-space model*: a rule that assigns to every point of a state space the instantaneous rate at which the system moves through it. Throughout this thesis we restrict attention to the class of *linear time-invariant* (LTI) systems, for which this rule is linear in both the state and the input and does not itself depend on time[CITE: ControlSystemsTheoryAndDesign.pdf].

A continuous-time LTI system is given by

$$\dot{x}(t) = A_c\, x(t) + B_c\, u(t), \qquad x(0) = x_0, \tag{2.1a}$$

$$y(t) = C\, x(t) + D\, u(t), \tag{2.1b}$$

where $x(t) \in \mathbb{R}^n$ is the *state vector*, $u(t) \in \mathbb{R}^m$ the *input vector* chosen by the agent, and $y(t) \in \mathbb{R}^q$ the *output vector*, i.e. the output observed and thus available to the feedback controller. Accordingly, $A_c \in \mathbb{R}^{n\times n}$ is called the *system matrix*, $B_c \in \mathbb{R}^{n\times m}$ the *input matrix*, $C \in \mathbb{R}^{q\times n}$ the *output matrix*, and $D \in \mathbb{R}^{q\times m}$ the *feedthrough matrix*.

The two equations in (2.1) play fundamentally different roles, and it is helpful to explain their distinction. Equation (2.1a) is a differential equation: it does not

compute the state, which is assumed to be known at every instant, but prescribes its *rate of change*. The pair (A_c, B_c) therefore governs how the system *behaves*, with A_c describing its autonomous dynamics and B_c describing how the input of the agent influences them. Equation (2.1b), in contrast, contains no derivative at all. It is an instantaneous readout, and the pair (C, D) therefore describes how the system is *observed* rather than how it evolves. We set $D = 0$ throughout, which is the standard assumption for systems in which the actuator has no direct instantaneous influence on the sensor[CITE: either same notes or textbook].

Given an input signal $u(\cdot)$, the trajectory of (2.1a) is obtained by integration and is given in closed form by the variation-of-constants formula

$$x(t) = e^{A_c t} x_0 + \int_0^t e^{A_c(t-s)} B_c \, u(s) \, \mathrm{d}s, \tag{2.2}$$

where $e^{A_c t}$ denotes the matrix exponential[CITE: same textbook]. This makes the interpretation of A_c precise: in the scalar case $\dot{x} = ax$ yields $x(t) = e^{at} x_0$, so a is a growth *rate* rather than a value. Equation (2.2) generalizes this, with the eigenvalues of A_c acting as growth rates along the corresponding eigendirections.

Full-state measurement. Generally, we assume that the state is fully measurable and therefore set $C = I_n$, which reduces (2.1b) to $y(t) = x(t)$. However, when we introduce disturbances through noise, this is no longer the case. More details on this are given in Section 2.3.1. The distinction between $u = -Kx$ and $u = -Ky$ is therefore critical, since the former admits an optimal solution while the latter might not; we return to this point in Section 2.1.4.

2.1.1.1 Discretization

Digital controllers, as in our case, do not act continuously: they sample the output at a fixed rate and hold the computed control input constant until the next sampling point. The learning algorithms studied in this thesis operate on such sampled trajectories as well, so our model has to reflect this. The standard approach is the *zero-order hold* (ZOH)[CITE: find reference]. Fixing a sampling period $h > 0$ and writing $x_k \coloneqq x(kh)$, the input is assumed to be piecewise constant,

$$u(t) = u_k \qquad \text{for } t \in [kh,\, (k+1)h), \quad k \in \mathbb{N}_0. \tag{2.3}$$

Substituting (2.3) into (2.2) over a single sampling interval and applying the change of variables $\tau = (k+1)h - s$ yields the discrete-time model

$$x_{k+1} = A_d \, x_k + B_d \, u_k, \tag{2.4}$$

with

$$A_d = e^{A_c h}, \qquad B_d = \left(\int_0^h e^{A_c \tau} \, \mathrm{d}\tau \right) B_c. \tag{2.5}$$

Two clarifications need to be made. First, (2.4) is not an approximation: it reproduces the continuous-time trajectory exactly at the sampling instants, provided the input is genuinely piecewise constant. This distinguishes ZOH from other schemes such as the forward Euler discretization $A_d \approx I + hA_c$, $B_d \approx hB_c$, which is only first-order accurate.[1] Second, the left-hand side of (2.4) is the actual next state rather than a derivative. The identity $(x_{k+1} - x_k)/h \approx \dot{x}(kh)$ recovers the continuous picture, and the identity block contained in A_d but not in A_c is exactly the state carried over from the previous step.

Unless stated otherwise, all experiments in this thesis are carried out in discrete time with sampling period $h = 0.01\,\mathrm{s}$, and we drop the subscript d in the following, writing simply A and B.

2.1.2 The Double Integrator

To make the comparisons across all chapters meaningful, all experiments are built on a single system: the *double integrator*, a frictionless point mass of mass m moving along a line under an applied force u[CITE: find some reference]. Its dynamics are Newton's second law,

$$m\,\ddot{p}(t) = u(t), \tag{2.6}$$

where $p(t)$ denotes the position of the mass. We simplify by using $m = 1$, so that force and acceleration coincide numerically. Choosing position and velocity as the state, $x = [\,p,\ v\,]^\top \in \mathbb{R}^2$, the second-order equation (2.6) decomposes into the first-order system

$$\dot{p}(t) = v(t), \qquad \dot{v}(t) = u(t), \tag{2.7}$$

in which the first equation is the definition of velocity and the second is (2.6) itself. In the notation of (2.1) this gives

$$A_c = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = I_2, \qquad D = 0. \tag{2.8}$$

The entry $(A_c)_{12} = 1$ should be read as stated in Section 2.1.1: the rate of change of position is given by the current velocity.

Because $A_c^2 = 0$, the matrix exponential in (2.5) truncates after two terms, $e^{A_c h} = I + A_c h$, and the discretization is available in closed form:

$$A_d = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} h^2/2 \\ h \end{bmatrix}. \tag{2.9}$$

This system is the minimal nontrivial choice for our purposes. It is second-order, so the state is not directly actuated and the controller must account for momentum; it is controllable for every $h > 0$; and it is not open-loop stable, since A_c has a double

[1] For the double integrator of Section 2.1.2 the two approaches happen to agree on A_d but *not* on B_d, where forward Euler gives $[0,\ h]^\top$ instead of the correct $[h^2/2,\ h]^\top$.

eigenvalue at the origin and A_d correspondingly a double eigenvalue at 1. The last property matters for the learning experiments: an uncontrolled or badly initialized policy diverges, so the assumption of a known stabilizing initial gain (Section 2.1.4) is a real restriction. These properties are made precise in Section 2.1.3.

2.1.3 Characteristics of a System

Now that we have defined our system, before we can optimize over it, it is important to clear up different characteristics of the same. The three we need are properties of the model and not of any particular controller. Stability is what the controller has to establish in the first place, controllability is what guarantees that it can, and observability is what allows a controller that only sees y to act as if it actually recorded x.

2.1.3.1 Stability

The first of these is stability, which guarantees that deviations in the system do not become too drastic, and thus cause some kind of failure. Mathematically, stability is a statement about a *solution* of the recursion $x_{k+1} = f(x_k, k)$, and is defined the following way [CITE: Astrom & Wittenmark, "Computer-Controlled Systems", p.77]:

Definition 2.1 (Stability)**.** Let $x^0(\cdot)$ be a solution of $x_{k+1} = f(x_k, k)$. The solution $x^0(\cdot)$ is called *stable* if for every $\varepsilon > 0$ there exists a $\delta(\varepsilon, k_0) > 0$ such that every solution $x(\cdot)$ with $\|x_{k_0} - x^0_{k_0}\| < \delta$ satisfies

$$\|x_k - x^0_k\| < \varepsilon \qquad \text{for all } k \geq k_0.$$

It is called *asymptotically stable* if it is stable and in addition $\|x_k - x^0_k\| \to 0$ as $k \to \infty$.

In the definition, ε bounds how far a perturbed trajectory is allowed to deviate, and δ says how small the initial distance must be to stay inside that bound. Asymptotic stability additionally requires the deviation to vanish with increasing time.

For linear systems this simplifies considerably. If $x(\cdot)$ and $x^0(\cdot)$ both solve $x_{k+1} = Ax_k$, then their difference solves the same recursion, so the behavior of *one* solution decides the behavior of all of them. Stability is thus a property of the system, explicitly the matrix A. This is stated as a theorem by [CITE: Astrom & Wittenmark, "Computer-Controlled Systems", p

Theorem 2.1 (Asymptotic stability of discrete-time LTI systems)**.** *The discrete-time LTI system $x_{k+1} = Ax_k$ is asymptotically stable if and only if all eigenvalues of A lie strictly inside the unit disk, i.e.*

$$\rho(A) := \max_i |\lambda_i(A)| < 1.$$

Our double integrator sits precisely on the boundary. From (2.9), A_d has the double eigenvalue $\lambda_{1,2} = 1$, so $\rho(A_d) = 1$ and Theorem 2.1 fails. It is not only

non-asymptotically stable but actually unstable. We can see that in the matrix powers

$$A_d^k = \begin{bmatrix} 1 & kh \\ 0 & 1 \end{bmatrix}, \tag{2.10}$$

which are growing without bound in k. This is also logical from a physical point of view as a frictionless mass with a nonzero initial velocity continues its trajectory forever if nothing acts on it. Every controller we construct therefore has to *create* stability rather than preserve it.

2.1.3.2 Controllability

Whether a system is controllable or not indicates if, independently of the controller, it is possible to reach any state inside its state space in a given time. A more formal definition is given in [CITE: ControlSystemsTheoryAndDesign]:

Definition 2.2 (Controllability)**.** The system is said to be *controllable* if for any initial state x_0 and any final state x_f there exist a horizon N and inputs $u_0, \dots, u_{N-1}$ such that the solution of (2.4) satisfies $x_N = x_f$. Otherwise, the system is said to be *uncontrollable*.

To check whether a given system is actually controllable, we need to compute the *controllability matrix* [CITE: ControlSystemsTheoryAndDesign]

$$\mathcal{C}(A, B) = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix} \in \mathbb{R}^{n \times nm}, \tag{2.11}$$

and then verify that it has full rank n; the system is controllable if and only if it does [CITE: ControlSystemsTheoryAndDesign]. Note that (2.11) is stated for a general input matrix B; for a single input ($m = 1$) it reduces to the square matrix $\begin{bmatrix} b & Ab & \cdots & A^{n-1}b \end{bmatrix}$.

Applying this to our double integrator, we obtain

$$\mathcal{C}(A_d, B_d) = \begin{bmatrix} B_d & A_dB_d \end{bmatrix} = \begin{bmatrix} h^2/2 & 3h^2/2 \\ h & h \end{bmatrix}, \qquad \det \mathcal{C}(A_d, B_d) = -h^3, \tag{2.12}$$

which is nonzero for every sampling period $h > 0$. The system is therefore controllable at every sampling rate, as claimed in Section 2.1.2.

For Section 2.1.4 we do not need the strong Definition 2.2 but can rather suffice with the weaker notion of *stabilizability* [CITE: ControlSystemsTheoryAndDesign]:

Definition 2.3 (Stabilizability)**.** The pair (A, B) is called *stabilizable* if there exists a gain $K \in \mathbb{R}^{m \times n}$ such that $\rho(A - BK) < 1$, i.e. if the closed loop obtained from (2.4) with $u_k = -Kx_k$ can be made asymptotically stable. Equivalently, (A, B) is stabilizable if and only if every mode of A that is not controllable is already asymptotically stable.

However, to decide whether our system is stabilizable, we do not need to construct this specific K. The Popov–Belevitch–Hautus (PBH) test [CITE: TODO: Find fitting] allows us to employ it as a rank condition on the relevant eigenvalues:

Theorem 2.2 (PBH test for stabilizability)**.** *The pair (A, B) is stabilizable if and only if*

$$\operatorname{rank} \begin{bmatrix} A - \lambda I_n & B \end{bmatrix} = n \qquad \text{for all } \lambda \in \mathbb{C} \text{ with } |\lambda| \geq 1. \tag{2.13}$$

Comparing (2.13) with (2.11) shows that the latter applies the same rank condition as the former, however imposed for *every* $\lambda \in \mathbb{C}$. Thus, controllability implies stabilizability but not vice versa. The difference contains systems which have uncontrollable directions decaying on their own, and a controller can hence never influence.

As our double integrator is already controllable by (2.12), it is therefore also stabilizable.

2.1.3.3 Observability

Finally, we need to take a look at observability. Above we assumed that we can actually observe the state of the system we are modeling. However, it is much more realistic that we only collect a recording of it, for example through sensors, and need to estimate the real state of the system. The notion of observability describes whether we can determine this state from our control input u and our recorded response y alone [CITE: ControlSystemsTheoryAndDesign].

Definition 2.4 (Observability)**.** The system with state-space model (2.4) and output $y_k = Cx_k$ is said to be *observable* if the initial state x_0 can be uniquely determined from the time history of the input u_k and the output y_k over a finite interval. Otherwise, the system is said to be *unobservable*.

Dual to the controllability matrix, we can derive the *observability matrix*

$$\mathcal{O}(C, A) = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{qn \times n}, \tag{2.14}$$

and make use of the following theorem [CITE: ControlSystemsTheoryAndDesign]:

Theorem 2.3 (Observability rank condition)**.** *The system is observable if and only if the observability matrix $\mathcal{O}(C, A)$ has full rank n.*

Using Theorem 2.3 for our double integrator is actually quite simple: we set $C = I_2$ in (2.8), so $\mathcal{O}(C, A_d)$ already contains the identity in its first block row and has full rank. The system is thus observable.

If we cannot uniquely determine x_k from our observation, we work with an estimate $\hat{x}_k$ instead. Running a copy of the model, $\hat{x}_{k+1} = A\hat{x}_k + Bu_k$, the estimation error $\tilde{x}_k \coloneqq x_k - \hat{x}_k$ follows the dynamics

$$\tilde{x}_{k+1} = A\,\tilde{x}_k, \qquad \tilde{x}_0 = x_0 - \hat{x}_0, \tag{2.15}$$

which is again just the system without control input. The decay of the error therefore depends only on the eigenvalues of A. If the system is unstable, the error grows infinitely [CITE: ControlSystemsTheoryAndDesign] and for our example it does so linearly, by (2.10). Observability fixes this by correcting the model with the measurement, $\hat{x}_{k+1} = A\hat{x}_k + Bu_k + L(y_k - C\hat{x}_k)$, which turns (2.15) into $\tilde{x}_{k+1} = (A - LC)\tilde{x}_k$. If the system is observable, we can choose an L such that the eigenvalues of $A - LC$ lie inside the unit disk, making it stable.

As with stabilizability, the weaker requirement is called *detectability*, and it is the condition we rely on in Section 2.1.4.

Definition 2.5 (Detectability)**.** The pair (A, C) is called *detectable* if there exists an observer gain $L \in \mathbb{R}^{n\times q}$ such that $\rho(A - LC) < 1$. Equivalently, (A, C) is detectable if and only if every unobservable mode of A is already asymptotically stable.

Detectability is the dual of Definition 2.3 which gives us that (A, C) is detectable if and only if $(A^\top, C^\top)$ is stabilizable [CITE: ControlSystemsTheoryAndDesign]. This follows as $\rho(A - LC) = \rho\big(A^\top - C^\top L^\top\big)$. Transposing the test for stabilizability (2.13) leaves us with the dual rank test:

Theorem 2.4 (PBH test for detectability)**.** *The pair (A, C) is detectable if and only if*

$$\operatorname{rank}\begin{bmatrix} A - \lambda I_n \\ C \end{bmatrix} = n \qquad \text{for all } \lambda \in \mathbb{C} \text{ with } |\lambda| \geq 1. \tag{2.16}$$

With $C = I_2$ the matrix in (2.16) contains an identity block and has full rank for every λ, so our system is observable and therefore detectable. The reason we introduce the weaker notion here is Theorem 2.5, whose hypothesis is the detectability of $(A, Q^{1/2})$ rather than (A, C). This shifts the focus from what the sensor sees to which directions of the state space are penalized by the cost.

The topic of observability becomes important again in Section 2.3.1, where we deliberately introduce noise so that $y_k \neq x_k$. This does not change our matrix $C = I_2$, so the rank condition of Theorem 2.3 still holds and the system remains observable by definition. However, y_k is no longer a deterministic function of x_k, so we can never estimate the state exactly. We revisit this and its implications there.

2.1.4 The Linear Quadratic Regulator

Now that we have defined our state-space model and its important characteristics, it is time to introduce our controller, which is based on the *linear quadratic regulator* (LQR) problem.

Here, the setting is that our system starts at an arbitrary initial state in its state space, and the goal of our controller is to return to the zero state as fast as possible. At the same time, the control input should be limited to a realistic quantity, such that it could actually be produced by an actuator. Both requirements can be combined into a single cost function, where Q describes the former, while R associates costs to control input. Over a finite horizon T, this gives the cost [CITE: OptimalAndRobustControl.pdf]

$$V_T(x_0, u) = \int_0^T \left(x(t)^\top Q\, x(t) + u(t)^\top R\, u(t)\right) \mathrm{d}t, \qquad Q = Q^\top \succeq 0, \quad R = R^\top \succ 0. \tag{2.17}$$

We notice two further restrictions here: Q is required to be positive semi-definite, meaning deviating from the origin is not reducing costs, and R must be positive definite, such that every control input results in a cost.

As the original formula relates to a continuous problem with a fixed end time T, it does not fit our goal well of evaluating over an infinite discrete horizon. The following formula offers exactly that:

$$J(x_0, u) = \sum_{k=0}^{\infty} \left(x_k^\top Q\, x_k + u_k^\top R\, u_k\right). \tag{2.18}$$

This sum does not necessarily converge. If the closed loop does not decay, the costs do not vanish and J tends to $+\infty$. Using this as our definition, J becomes well defined for every gain.

2.1.4.1 The Optimal Gain

What makes the LQR problem special is that its solution is a static linear feedback law: the optimal input at time k depends only on the current state of the model. Introducing a gain matrix $K \in \mathbb{R}^{m \times n}$, we write it as [2]

$$u_k = -K x_k. \tag{2.19}$$

Our controller is thus reduced to choosing a matrix, and the optimal controller is given by the optimal gain matrix K^*.

As we are dealing with a discrete-time system, that gain is obtained from the *discrete algebraic Riccati equation* (DARE)

$$P = A^\top P A - A^\top P B \left(R + B^\top P B\right)^{-1} B^\top P A + Q, \tag{2.20}$$

[2] Note that the minus in front of K has been chosen as the convention for this thesis. This turns our closed loop into the format $A - BK$, while some referenced literature uses $A + BK$. Formulas cited have been adapted to our convention by replacing K with $-K$.

whose stabilizing solution $P = P^\top \succeq 0$ yields

$$K^* = \left(R + B^\top P B\right)^{-1} B^\top P A, \qquad u_k^* = -K^* x_k. \tag{2.21}$$

The matrix P is additionally our *cost-to-go* matrix, meaning that we can use it to calculate the optimal cost incurred from any state x onwards with

$$J^*(x) = x^\top P x. \tag{2.22}$$

Whether (2.20) has such a solution at all is answered by the structural properties of Section 2.1.3 [CITE: OptimalAndRobustControl.pdf]:

Theorem 2.5 (Existence and uniqueness)**.** *Let (A, B) be stabilizable and let $(A, Q^{1/2})$ be detectable. Then the DARE (2.20) has a unique symmetric positive semi-definite solution P, which is the only stabilizing one, and the resulting closed loop satisfies $\rho(A - BK^*) < 1$. If $(A, Q^{1/2})$ is observable, P is positive definite.*

2.1.4.2 The Cost as a Function of the Gain

Equation (2.21) solves the problem, but it solves it directly and tells us nothing about the quality of non-optimal gains. The learning algorithms of the following chapters never see the model itself; they search over gains directly, and for that we need the cost as a function of K rather than of an input sequence.

Fix a gain K and write the resulting closed loop as $x_{k+1} = A_K x_k$ with $A_K \coloneqq A - BK$. Substituting (2.19) into (2.18) shows that the cost is again a quadratic form, $J(x_0, K) = x_0^\top P_K x_0$, where P_K now solves the far simpler discrete *Lyapunov equation*

$$P_K = A_K^\top P_K A_K + Q + K^\top R K, \tag{2.23}$$

which is linear in the unknown P_K and has a unique solution whenever $\rho(A_K) < 1$. This restricts us to the set of stabilizing gains

$$\mathcal{K} \coloneqq \left\{K \in \mathbb{R}^{m \times n} : \rho(A - BK) < 1\right\}, \tag{2.24}$$

outside of which we set the cost to $+\infty$, consistent with the remark after (2.18).

One dependency remains, namely on the initial state. Evaluating the gain based on a single x_0 would reward gains that happen to fit that one point, so we instead draw the initial state from a distribution, $x_0 \sim \mathcal{D}$, and average. Writing $\Sigma \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}}[x_0 x_0^\top]$ for its second moment, the objective becomes

$$f(K) \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}}\left[x_0^\top P_K x_0\right] = \operatorname{tr}\left(P_K \Sigma\right), \tag{2.25}$$

using the cyclic property of the trace. Equation (2.25) is the object that every remaining chapter of this thesis minimizes, and the algorithms differ only in how much of the model is known to them.

Throughout this thesis we set $Q = I_2$ and $R = 1$, which weighs position, velocity and control effort equally. Initial states are drawn uniformly from the box $[-10, 10] \times [-1, 1]$.

2.2 Motivation

An agent acting in its environment receives observational input and responds to it with a commanded control input. In other words, the agent is a function in relation to its environment, and to find an optimal behavior, we need to approximate this optimal function as closely as possible. While neural networks are known to be universal approximators [4, 12], and hence able to approximate any function, in the case of an embodied agent, different constraints need to be taken into account. Ay [2] names additionally to external embodiment constraints possible intrinsic constraints, and argues that such can even be "beneficial" for the control structure. Additionally, fewer parameters for deployment are helpful due to energy usage and storage capacity, the core of the notion of cheap design[CITE: TODO]. So, for such embodied agents, the goal is to find an "embodied universal approximator" [23, p. 3], which restricts the neural network architecture to an intrinsic, unknown dimension, encoding physical properties belonging to the agent and the environment, while disallowing impossible movements [23, Sec. 3].

Biology motivates this approach even further: here it is clear to see that by commanding a body, some features of the control architecture can be simplified and delegated to the embodiment[CITE: TODO]. For example, McGeer's passive walker [21] clearly shows that some movements can be produced by the body without any control input, just by choosing the right embodiment. While obviously here being simplified to not include any control at all, it can quite simply be extended with such[CITE: #9, McGeer reference to his previous paper]. Müller and Hoffmann [24, p. 4] mention such a combination, a robot climbing up walls, whose design was inspired by geckos, indeed using energy for control, however very little compared to an approach not exploiting this specific embodiment design. Further, it can be seen that the complex behaviors are not only less energy-intensive to produce, but also utilize muscle synergies [6] and thus fewer control parameters. And while it is difficult to clearly attribute the effect of the embodiment [41], for us, this gives reason to believe that the simplicity, and thus efficiency, comes from a low-dimensionality of the controller [23].

That this is indeed true for certain examples has been shown both in theory and practice before [5, 22, 19]. Li et al. [19] call this lower dimension "intrinsic dimension", which is the dimensionality required for the model to be able to find policies. They show, for example, that for achieving 90 % of the performance of a fully connected network on MNIST, the dimensionality can be reduced by 99.6 %. For a convolutional neural network, it is still a 99.3 % reduction. Others show that the "least complex model which is able to learn an optimal policy does learn faster" [22]. For a neural network, reducing parameters can be done in different ways[CITE: TODO], some of which we look at in Chapter 4. In comparison, for LQR, where the controller is a gain matrix, the corresponding approach is reducing its rank [30]. However, while these approaches are more parameter-efficient while achieving similar or slightly worse performance, it was also observed that they tend to perform even worse when encountering noise [5].

Robustness to noise, however, is one of the important characteristics of controllers acting under real-world conditions [28, 31]. One approach of achieving this in nature is the principle of "redundancy and diversity" [16], which stands in direct contrast to our goal of reducing redundant parameterizations. Another reason for producing robust controllers is the difficult transfer from digital training environment to real-world embodiments, commonly referred to as the "reality gap" [36, 26]. Approaches of training robust controllers typically introduce noise already during training, be it either by using a linear quadratic Gaussian (LQG) controller or by including variations to the dynamics [28, 26]. These approaches, however, do not look at low-dimensionality, which makes sense as Kitano [16, p. 833] poses that robustness cannot be increased without sacrificing performance or accepting "increased resource demands".

Before taking a look at the more complex case of RL, we start simpler by using the simpler and better-understood LQR. This approach has been taken before [29] and does make sense given the guarantees regarding convergence and effectiveness of policy optimization for it [11, 34], as well as the understanding of its geometry [35]. It also offers methods which produce certain structures in its controller, like sparsity or our desired low-dimensionality, as given by structured policy iteration (S-PI) [25]. Approaches can generally be categorized into two groups if the underlying dynamics are unknown and only data is available: estimating the model and then applying a model-based solver (indirect), or using the data to estimate the gain matrix (direct) [43]. Both the indirect, as in system identification with uncertainty bounds [7], and the direct setting, with an $\mathcal{H}_\infty$ constraint that bounds the closed-loop gain from disturbance to output [42], have been studied with regard to robustness, however, to our knowledge, not in combination with low-rank policies.

Research evaluating RL on control tasks has also shown that there is room for simplification, for example using linear policies instead of complex, fully connected neural networks [20, 27]. This, together with the fact that parameters in neural networks are highly dependent, and thus redundant [9], allows us to assume that low-dimensionality can improve efficiency here. Examples of this exist, for example by factorizing a discretized low-rank policy into a thin and wide matrix [32] or by introducing an orthogonal projection inside the layer structure [37]. While they produce lower dimensionality, a different approach uses "information bottlenecks", which exhibit better generalization behavior in unseen environments [13, Sec. 4.2], which again could help us with robustness.

A clear case where low-dimensional dynamics underlie a high-dimensional observational space is in perception-based tasks. Agents observing only images instead of the states of simple control tasks are posed with the challenge of extracting these states from a high-dimensional, often nonlinear encoding [39, Sec. 1]. Many approaches, based on state representation learning (SRL), exist, which use different variants to create a latent embedding of the image, used as a state [17]. The most interesting approach, especially for us looking at embodied agents, is the usage of

so-called "priors", which restrict the latent space by using knowledge about the environment [17]. This brings us back to Ay's argument that the embodiment actively defines the action space [2]. Robotic priors were introduced by Jonschkowski and Brock [14] in 2015, and then further refined by Botteghi et al. [3] for continuous action spaces. A more specific version restricted to position and velocity also exists here, training a model to encode an image into a low-dimensional position state, from which the velocity is calculated by finite differences [15]. While all that sounds promising, in contrast stand findings hinting that the problems of robust control also apply to perception-based control, as noise induced through perception directly affects the performance [40] and thus needs approaches for improving it [8].

Our goal for this thesis is therefore to look a bit deeper at the combination of low-dimensionality and robustness in an empirical way, motivated by the findings of Datar and Ay [5]. LQR serves as the perfect, well-investigated basis [29], offering low-rank approaches that have been proposed and analyzed theoretically, but have not yet been applied in practice and under noise [25]. Previously investigated policies for LQR under noise in comparison did not look at low-dimensionality specifically [42]. For RL, low-dimensionality has been explored previously [1, 33], and also in comparison with robustness [18], but does not take into account the embodiment we are interested in. Closing this gap, our main interest is to compare robustness properties for low-dimensional policies for embodied agents under increasingly more complex and noisier environments.

2.3 Approach

2.3.1 Uncertainty and Disturbance Models

Until now, we assumed that we can fully observe the system, and that its behavior is governed by exactly the dynamics we know. However, in reality there are influences on such systems which we do not clearly know of. Especially for embodied agents this is important, as the world is a messy place.

We thus introduce four different disturbances to be included in the model, commonly known as noise. They are distinguished by the point at which they affect the loop: what the controller *observes* (measurement noise), what its output actually *does* (control noise), what the plant *is* (parametric uncertainty), and what else *acts* on it (external disturbance). Each of them can be activated separately, so that we can clearly isolate the cause of an effect. Throughout this section k indexes the sampling instants, and σ denotes the standard deviation of the respective noise.

2.3.1.1 Measurement Noise

This one mainly relates to observability: we influence the sensors used to measure the model state, such that capturing the same state twice would not lead to the

same observation y_k. Let n_k^{m} denote the measurement noise at step k. Then

$$n_k^{\mathrm{m}} \sim \mathcal{N}\big(0, \sigma_{\mathrm{m}}^2 I_n\big)\,, \qquad y_k = x_k + n_k^{\mathrm{m}}, \qquad \sigma_{\mathrm{m}} \in \mathbb{R}_+, \tag{2.26}$$

drawn independently at every step.

While one might think that this noise removes observability for us, it does not. The output matrix is still $C = I_2$, so the rank condition of Theorem 2.3 holds unchanged and observability is given.

However, it does remove the deterministic link between state and output, so we cannot identify x_k exactly with any amount of measurements. Using our noisy observation in the feedback law (2.19) and $u_k = -Ky_k$, the closed loop becomes

$$x_{k+1} = (A - BK)\, x_k - BK\, n_k^{\mathrm{m}}. \tag{2.27}$$

So even a perfectly stabilizing K can no longer bring the state exactly to the origin, but rather to a distribution around it.

2.3.1.2 Control Noise

Control noise, as the name says, acts on the applied control input. With it, we try to model the difference between the intended movement of an actuator and its actual action due to different factors. Here we deal with *multiplicative* noise, which scales with the input. In detail this means that the applied control input $\tilde{u}_k$ is obtained as

$$n_k^{\mathrm{c}} \sim \mathcal{N}\big(0, \sigma_{\mathrm{c}}^2 I_m\big)\,, \qquad \tilde{u}_k = (1 + n_k^{\mathrm{c}}) \odot u_k, \tag{2.28}$$

where $\odot$ denotes the elementwise product, so that σ_{c} is the *relative* error. Applying this to the input, the next step becomes

$$x_{k+1} = A\, x_k + B\, \tilde{u}_k. \tag{2.29}$$

2.3.1.3 Parametric Uncertainty

This noise acts directly on the matrices of the model. Here, we either decide to keep it time invariant, meaning $\Delta_k = \Delta_0$ for all k, or to resample it at every step. The former models a wrongly identified system, while the latter simulates one that continuously changes. As with measurement noise, it is additive. Applied, it acts on the system the following way:

$$\big(\Delta_k^A\big)_{ij}\,,\ \big(\Delta_k^B\big)_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}\big(0, \sigma_{\mathrm{p}}^2\big)\,, \qquad x_{k+1} = \big(A + \Delta_k^A\big)\, x_k + \big(B + \Delta_k^B\big)\, \tilde{u}_k. \tag{2.30}$$

Worth noting is that resampling every step leaves the model correct *on average*, since $\mathbb{E}[\Delta_k] = 0$, and mainly adds variance to the trajectory. This stands in contrast to the time-invariant version, where we have a discrepancy between what we compute actions on, and what actually happens.

2.3.1.4 External Disturbance

This noise models external influences, like a constant wind pushing on our agent, or periodic disturbances, for example rain hitting it with some frequency. It only affects the position, so the noise is $n_k^{\mathrm{d}} = d_k\, e_1 \in \mathbb{R}^n$ where e_1 is the first unit vector. Depending on the type, the push intensity d_k is drawn as

$$\text{Gaussian:} \quad d_k \sim \mathcal{N}\big(0, \sigma_{\mathrm{d}}^2\big) \quad \text{independently at every step,} \tag{2.31a}$$

$$\text{Sinusoidal:} \quad d_k = \sigma_{\mathrm{d}} \sin(0.01\, k), \tag{2.31b}$$

$$\text{Constant:} \quad d_k = d_0 \sim \mathcal{N}\big(0, \sigma_{\mathrm{d}}^2\big) \quad \text{for all } k. \tag{2.31c}$$

Only the first one can really be considered as noise; the others are more deterministic values, where σ_{d} acts as the amplitude of it. The sinusoidal variant completes one period every $2\pi/0.01 \approx 628$ steps, which at a sampling interval of $h = 0.01\,\mathrm{s}$ is about 6.3 seconds, so rather a slow drift than a rapid vibration. As the disturbance acts on the true state x_k, it also enters before any lifting of the observation in Chapter 4. Including the noise in our loop, the state transition becomes

$$x_{k+1} = A\, x_k + B\, \tilde{u}_k + n_k^{\mathrm{d}}. \tag{2.32}$$

2.3.1.5 The Full Loop

While we, until now, have only regarded each noise type on their own, there is also the possibility to include them all at once. Doing so, a single step becomes:

$$y_k = x_k + n_k^{\mathrm{m}}, \tag{2.33a}$$

$$u_k = -K y_k, \tag{2.33b}$$

$$\tilde{u}_k = (1 + n_k^{\mathrm{c}}) \odot u_k, \tag{2.33c}$$

$$x_{k+1} = \big(A + \Delta_k^A\big)\, x_k + \big(B + \Delta_k^B\big)\, \tilde{u}_k + n_k^{\mathrm{d}}. \tag{2.33d}$$

Relevant to note about (2.33) is that the controller only interacts with y_k and u_k, whereas the dynamics and the stage cost of (2.18) are evaluated on the true state x_k. Thus our controller can never actually observe the true state it is evaluated on. For our baseline LQR solution of Section 2.1.4, which we use for comparison, we compute the gain on a pure system, and only afterwards apply it to an *exposed* system of the style (2.33). Measuring how much performance this difference makes is the subject of the next section.

2.3.2 Metrics and Robustness

In this thesis we try to answer two main questions. First, how well a trained controller performs generally, as well as under influences unknown to it during training. The second relates more to the size of the controller, and a possible trade-off between performance and the former.

2.3.2.1 Cost and Suboptimality

By Theorem 2.5 no gain achieves a lower cost (2.25) than K^*, and every controller we train is therefore in the best case only as good as it. This serves as our baseline, and we evaluate the cost produced by our controllers in relation to it.

While we can easily calculate the infinite-horizon cost of a gain via the Lyapunov equation (2.23), we cannot effectively use it. The reason for that is that (2.23) can only be applied to the closed loop, however when adding noise to our evaluation, this is not the case anymore. Additionally, the neural networks we train in later chapters do not admit a gain-matrix-based policy P_K, which would need a different approach anyway.

So, we instead evaluate them over a fixed horizon via rollout. Hence for a given controller, a noise condition c, which is a subset of channels and an intensity (Section 2.3.2.3), and a horizon T, we simulate starting from a randomly drawn $x_0 \sim \mathcal{D}$. The resulting loop (2.33) under c thus accumulates

$$J_T^{(i,s)}(K;c) = \sum_{k=0}^{T-1} \left(x_k^\top Q\, x_k + \tilde{u}_k^\top R\, \tilde{u}_k \right), \tag{2.34}$$

where s indexes the seed the run was trained under and i the episode within it.

Writing $\mathcal{F}$ for the pairs (s,i) whose episode stayed finite, the cost estimate becomes

$$\hat{J}_T(K;c) = \frac{1}{|\mathcal{F}|} \sum_{(s,i)\in\mathcal{F}} J_T^{(i,s)}(K;c), \qquad \mathcal{F} = \left\{ (s,i) \;:\; J_T^{(i,s)}(K;c) < \infty \right\}. \tag{2.35}$$

Our final metric is the relative excess of the cost produced by a trained controller over the cost of the optimal controller K^* under the same condition. It is taken *per episode*, meaning an episode (s,i) of a trained controller and episode (s,i) of the baseline are compared here. Both start from the same x_0 and are affected by the same noise. Overall this yields:

$$S^{(i,s)}(K;c) = \frac{J_T^{(i,s)}(K;c) - J_T^{(i,s)}(K^*;c)}{J_T^{(i,s)}(K^*;c)}, \qquad \hat{S}(K;c) = \frac{1}{|\mathcal{F}|} \sum_{(s,i)\in\mathcal{F}} S^{(i,s)}(K;c), \tag{2.36}$$

where $S = 0$ means that the trained controller performed as well as our baseline and $S = 0.1$ that it did 10 % worse.

Note that this metric can indeed be negative. K^* is only optimal for the nominal problem (Section 2.1.4), which has exact state feedback on known matrices and no external disturbance. These conditions do not hold, unless no noise is applied, which is only true if $c = \varnothing$.

2.3.2.2 Test Protocol

For all evaluation throughout this thesis, we use a standardized approach, so that only the trained controller varies between different runs.

- **Seeds.** $M = 10$ fixed seeds, drawn once randomly.
- **Episodes.** $N = 100$ rollouts per seed, each from an independently sampled $x_0 \sim \mathcal{D}$.
- **Horizon.** $T = 2000$ steps at $h = 0.01\,\mathrm{s}$, so a total of 20 seconds.
- **Initial states.** Drawn from the box $[-10, 10] \times [-1, 1]$ of Section 2.1.4.
- **Initial gain.** For iterative solvers requiring a stable initial gain, we use $K_0 = [\,0.1,\ 0.5\,]$, whose closed loop has $\rho(A - BK_0) \approx 0.9975$.

Under the stronger conditions individual rollouts might diverge. To keep our mean finite, we remove them from the mean calculation, and report them separately as the divergence rate:

$$D(K; c) \coloneqq \frac{1}{MN} \left| \left\{ (s, i) \ : \ J_T^{(i,s)}(K; c) = \infty \right\} \right|. \tag{2.37}$$

An individual episode can be missing from the result due to another reason. Where the *baseline* evaluation of episode (s, i) diverged, $S^{(i,s)}$ has no value at all, since (2.36) would divide by an infinity. Those episodes are excluded from $\hat{S}$ and named separately.

2.3.2.3 Noise Intensity

Section 2.3.1 introduced four channels which change the system compared to the base model. Each noise channel is separated further into five intensities,

$$\textit{none} \prec \textit{weak} \prec \textit{standard} \prec \textit{strong} \prec \textit{divergent},$$

where *none* leaves the system as it is and thus serves as our control condition. All the other intensities are calibrated by how much they increase the cost of the optimal gain K^* on the uninflated system. For each channel c and intensity ℓ, we choose the standard deviation in a way such that with the test protocol of Section 2.3.2.2

$$\hat{J}_T(K^*; c_\ell) = \rho_\ell \, \hat{J}_T(K^*; \varnothing), \qquad \rho_\ell \in \{1.2,\ 1.5,\ 3,\ 10\} \tag{2.38}$$

applies for *weak*, *standard*, *strong* and *divergent* respectively. For K^*, the increase stays the same under inflation for all channels except control noise. As it acts elementwise on the inflated output, it is averaged out over the number of actuators: *strong* control noise raises the cost of K^* by a factor of 3.0 without inflation, but only by 1.22 and 1.05 at the sizes $d_{\text{in}} \times d_{\text{out}} = 50 \times 25$ and 200×100. For simplicity, however, we do not vary the levels by size, but instead accept this as a side effect. The *divergent* intensity, contrary to its name, does not cause divergence of the baseline, but makes it ten times as costly. The concrete standard deviations per channel are listed in Table A.2.

After an initial analysis of the influence of the different channels separately and combined at their different intensities, we decide on one of them for the remaining chapters.

2.3.2.4 The Size of a Policy

The second question is one of size. For a neural network the known way is simply the number of weights and biases of the actor [CITE: TODO]. For a gain matrix however, we need to find a definition that is comparable to it.

The simplest approach for a $K \in \mathbb{R}^{m \times n}$ is the number of its entries, mn. In Chapter 3, however, we reduce the rank of K. Optimizing the storage size for K of rank r, the singular value decomposition [CITE: SVD_Notes.pdf] separates it into

$$K = U\Sigma V^\top, \qquad U \in \mathbb{R}^{m \times r}, \quad \Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r), \quad V \in \mathbb{R}^{n \times r}, \tag{2.39}$$

with U and V having orthonormal columns and $\sigma_1 \geq \cdots \geq \sigma_r > 0$ being the non-zero singular values. Hence, we only need to store mr entries for U, r for the diagonal of Σ and rn for V, which in total yields $r(m+n+1)$. For a small enough r this is less, so we take the minimum of both:

Definition 2.6 (Policy size)**.** The size of a linear policy $K \in \mathbb{R}^{m \times n}$ of rank $r = \operatorname{rank}(K)$ is

$$|K| \coloneqq \min\left\{mn,\ r\,(m+n+1)\right\},$$

the number of real numbers required to store it. The size of a neural policy is its total number of trainable parameters in the actor.

As mentioned before, the LQR serves as the perfect starting example, which is exactly why we start our investigation here. For our investigations, we will use the double integrator, as mentioned in Section 2.1.2.

First, as a basis, we will start by comparing the standard policy gradient (PG) approach [11] with S-PI [25] on the true model. Their relative performance will be evaluated

Afterwards, we will take a look at the cases where we do not have access to the real model. We compare the relative performance of both direct and indirect approaches to our baseline, which is the optimal controller calculated via the DARE (2.20) on the true model. For the indirect approaches, we will use numerical algorithms for subspace state space system identification (N4SID)[CITE: 60 VanOverschee] to estimate the underlying model from one trajectory. A more detailed description, as well as the exact parameters selected, will be provided in the sections discussing the results. With this estimated model as the basis, we employ three different approaches on how to learn our policy, called N4SID + DARE, N4SID + Model-based PG and N4SID + S-PI. All these will also be discussed in Chapter 3.

As we want to investigate higher-dimensional controllers with underlying low-dimensional dynamics, we need to create such a setup. Here, we choose to use orthonormal projection matrices, which can be used to transfer observations between the low-dimensional true state of the state-space model and the inflated one our controller sees. Conversely, for a high-dimensional actuation, we project into the lower-dimensional space via a second matrix. In detail, for an inflated input

dimension d_{in} and output dimension d_{out}, the two lifts are constructed the following way:

$$U_1 \in \mathbb{R}^{d_{\text{in}} \times n}, \quad U_1^\top U_1 = I_n, \qquad U_2 \in \mathbb{R}^{m \times d_{\text{out}}}, \quad U_2 U_2^\top = I_m, \tag{2.40}$$

where U_1 is the orthonormal factor of the QR decomposition of a random matrix, whose entries are sampled from the standard normal distribution. U_2 is the transpose of such a factor sampled from a separately drawn matrix. The controller now only sees the lifted observation and acts through the lifted action, so a single step of the loop becomes

$$y_k = U_1 x_k, \tag{2.41a}$$
$$v_k = -K y_k, \qquad K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \tag{2.41b}$$
$$u_k = U_2 v_k, \tag{2.41c}$$
$$x_{k+1} = A x_k + B u_k. \tag{2.41d}$$

The system is therefore controlled by the effective gain $U_2 K U_1 \in \mathbb{R}^{m \times n}$, whose rank is at most $\min(m, n)$.

For Chapter 3 and the following chapter, we will use these preset inflation sizes under the names given in Table 2.1.

Table 2.1: Preset inflation sizes.

Name	d_{in}	d_{out}
base	2	1
small	10	5
medium	50	25
large	200	100

Our first approach for Chapter 3 is to compare our selected methods across these inflation sizes, and see how increasing the dimension affects the performance of each. As the next step, we will introduce the different noise channels named in Section 2.3.1 first to the base double integrator, where no overparameterization exists. The results we observe here will then be compared with evaluations of the controllers for a system with *large* inflation under different noise channels and strengths.

In Chapter 4, we will then first introduce RL-based approaches. With proximal policy optimization (PPO)[CITE: TODO] being one of the most used variants in robotics[CITE: TODO], we choose this as our main architecture, and apply different variations of it. First, by doing xyz[CITE: TODO, missing text] we force the trained controller to be linear, which allows us to extract it as a gain matrix that we can closely compare with our LQR approaches (Linear PPO). The fully connected, standard

version is named Fully-connected PPO, and will also be evaluated. Additionally, we will introduce structures that force the observations through a low-dimensional latent space. Here, we will reuse the N4SID system identifier from before, by creating an encoding matrix from the singular value elimination, that projects the high-dimensional observation first into the low-dimensionality, and then trains PPO with this as the input. Therefore, the name for it is State encoder + PPO. A similar approach is used for Scalable encoder + PPO, which is based on Jonschkowski and Brock [14] and Botteghi et al. [3], and uses robotic priors to derive a low-dimensional latent state via a trainable encoder. Again, the construction following this encoder is PPO. Lastly, we implement the orthonormal bottlenecks introduced by Todorov et al. [37] (Bottleneck PPO). It consists of sampling a fixed orthonormal matrix B of rank k, which is added at the end of a trainable feature extractor, and used to project the output into a k-dimensional space. Like with the other encoders, we follow it with our PPO architecture.

All these low-dimensional approaches, if visualized, exhibit a "bow-tie" shape, which in nature can actually improve robustness [16]. So this is indeed interesting to investigate. However, first, like in Chapter 3, we start by comparing these different approaches across the different inflation sizes, without any noise applied, to see how they generally behave. Then, we introduce noise, and compare our approaches under *large* inflation dimensions.

Taking our results from the experiments on the artificially inflated dimensions, we create a new setup, still using the double integrator as our underlying structure. Here, we will observe the mass via a simulated camera, and place a number of actuators evenly distributed among angles on a half-circle. This will give us an actual high-dimensional input, as well as a high-dimensional output, that is in reality governed by low-dimensional dynamics.

By inputting images into our existing solvers, the dimensionality increases drastically. To be able to cleanly separate effects caused by the nonlinearity of the image construction from architecture-related effects, we will first artificially inflate our inputs to exactly the size that the images will be. Also, we set the output dimension to xyz[CITE: TODO], mimicking the setting we will use for the pixelated environment. This evaluation will only be done on Scalable encoder + PPO, Fully-connected PPO and Bottleneck PPO, as the other approaches either do not scale well or are not relevant to the comparison.

Then, using the same architectural constructions for these solvers, but applied to our new environment, we compare them with the additionally introduced position-velocity encoder [15] (Position-Velocity Encoder + PPO), which uses similar robotic priors to Scalable encoder + PPO, but focuses more on position, velocity and acceleration. It learns positional encodings from the observations, and then calculates the velocity state based on finite differences. Here, it is quite clear that we use knowledge of the limitations of the embodiment to construct more optimal controllers, again in a bow-tie shape.

Chapter 3

Robustness of LQR Solvers in Different Dimensions

3.1 A Known Model

We start by comparing two approaches that have knowledge of the underlying model, and thus do not need to estimate gradients but can calculate them exactly. This allows us to clearly separate the benefits of a low-dimensional controller from estimation errors.

3.1.1 Setting

Fazel et al. [11] introduced global convergence guarantees for policy gradient methods based on the true model in 2019. The paper describes a method for calculating the gradient of a policy from the system and cost matrices, and updating the policy accordingly. As the paper uses the same $u_k = -Kx_k$ convention as we do, we can implement the loop directly in the following way.

Starting from an initial, stabilizing gain $K_0 \in \mathcal{K}$ and the second moment Σ_0 of the initial state distribution, every iteration i first solves the discrete Lyapunov equation (2.23) for P_{K_i} using the implementation from SciPy [CITE: scipy]. The state correlation matrix Σ_{K_i} then follows from solving a second Lyapunov equation,

$$\Sigma_{K_i} = (A - BK_i)\Sigma_{K_i}(A - BK_i)^\top + \Sigma_0, \tag{3.1}$$

where, for initial states drawn uniformly from $[-10, 10] \times [-1, 1]$ as described in Section 2.1.4.2, $\Sigma_0 = \operatorname{diag}(10^2/3,\ 1^2/3)$. The policy gradient

$$\nabla f(K_i) = 2\left(\left(R + B^\top P_{K_i} B\right) K_i - B^\top P_{K_i} A\right) \Sigma_{K_i} \tag{3.2}$$

is then used for a gradient step on the gain matrix. Algorithm 1 summarizes the loop.

Algorithm 1 Model-based PG

Require: stabilizing gain $K_0 \in \mathcal{K}$, second moment Σ_0, step size η

1: **for** $i = 0, 1, 2, \dots$ **do**
2: solve (2.23) for P_{K_i}
3: solve (3.1) for Σ_{K_i}
4: compute the policy gradient $\nabla f(K_i)$ by (3.2)
5: update the gain by $K_{i+1} = K_i - \eta \nabla f(K_i)$
6: **end for**

The fixed learning rate η is a hyperparameter, which we set to $\eta = 10^{-5}$ for all inflation sizes, based on experiments whose results can be found in Section C.1. This policy gradient approach serves us as the basic, fully-parameterized implementation for this experiment.

Now we also want to implement a solver that produces a low-dimensional controller. As in our case the controller is characterized by the gain matrix, and for matrices dimensionality is "often [...] expressed by means of the rank" [30], we aim at learning a low-rank gain matrix. However, to make this comparable later to RL approaches that rely on the number of parameters, we measure the complexity of a gain by its policy size $|K|$ from Definition 2.6, which uses the more efficient version between the full and the factored parameterization.

Recht et al. [30] have already shown that nuclear norm minimization introduces low-rank solutions for convex matrix problems. However, as we are dealing with a non-convex optimization problem [11], we cannot transfer the results over directly. Here, Park et al. [25] offer us an approach to impose different structures on our policy during learning via regularizers based on norms, one of them being the nuclear norm. We implement the model-based approach introduced there with small adaptations in the following way.

We reuse the computation of the gradient $\nabla f(K_i)$ from Algorithm 1. For inducing the structure, we use an adaptive learning rate combined with a line search criterion. Starting from the initial step size $\eta = 1/(2\lambda)$, each iteration takes a gradient step and applies the proximal operator of the nuclear norm, which soft-thresholds the singular values,

$$K^+ = U \operatorname{diag}\left(\max\{\sigma_1 - \eta\lambda, 0\}, \dots, \max\{\sigma_r - \eta\lambda, 0\}\right) V^\top, \tag{3.3}$$

where $U \operatorname{diag}(\sigma_1, \dots, \sigma_r) V^\top$ is the singular value decomposition of the next policy candidate $K^+ = K_i - \eta \nabla f(K_i)$. The candidate K^+ is accepted if it is stabilizing, $\rho(A - BK^+) < 1$, and satisfies the sufficient decrease condition

$$f(K^+) \leq f(K_i) - \eta \operatorname{tr}\left(\nabla f(K_i)^\top G\right) + \frac{\eta}{2}\|G\|_F^2, \qquad G \coloneqq \frac{1}{\eta}\left(K_i - K^+\right). \tag{3.4}$$

Otherwise, the step size is reduced to $\beta\eta$ and the proximal step is repeated. The learning loop further can be stopped earlier if the criterion $\|K_{i+1} - K_i\|_F \leq \varepsilon$ is satisfied. Algorithm 2 summarizes the loop.

Algorithm 2 S-PI

Require: stabilizing gain $K_0 \in \mathcal{K}$, second moment Σ_0, weight λ, factor β, tolerance ε

1: **for** $i = 0, 1, 2, \ldots$ **do**
2: compute $\nabla f(K_i)$ as in Algorithm 1
3: set the initial step size $\eta = 1/(2\lambda)$
4: **for** $\ell = 1, \ldots, 20$ **do**
5: compute the candidate K^+ by (3.3)
6: **if** $\rho(A - BK^+) < 1$ and (3.4) holds **then**
7: **break**
8: **end if**
9: reduce the step size, $\eta \leftarrow \beta\eta$
10: **end for**
11: update the gain, $K_{i+1} = K^+$
12: **if** $\|K_{i+1} - K_i\|_F \leq \varepsilon$ **then**
13: **stop**
14: **end if**
15: **end for**

As one can see, there are a lot of parameters that can be modified here. We set $\varepsilon = 10^{-8}$, and use the same $\beta = 0.5$ as in the experiments section of Park et al. [25]. For λ, as it defines the learning rate as well, we have run an experiment with various values, which can be found in Section C.2. A value of $\lambda = 5$ has shown the most promising results across the sizes.

One thing that we implemented differently than the paper is the cap on the number of line search iterations in line four, which we set to 20. Some of our previous experiments had shown that if we leave it uncapped, for unstable updates which are impossible to stabilize, the loop runs until the update produces no effect at all.

Now that we have defined our controllers and how they learn, we test them in comparison to the DARE-calculated solution (2.20) from Section 2.1.4.1.

3.1.2 Results

We apply our solvers to the true models base and inflated versions, for now noise-free, as described in [CITE: cref inflation loop]. As described in [CITE: cref to test protocol], we train 10 controllers each per solver, before evaluating them on the test episodes. The results are then pooled. Let's first take a look at the training process at the uninflated system:

3.1.3 Noise Experiments

Now, we reuse the controllers we trained in [CITE: cref to subsec above] and evaluate them under the test protocol with noise[CITE: cref to noise introduction] injected. We only show

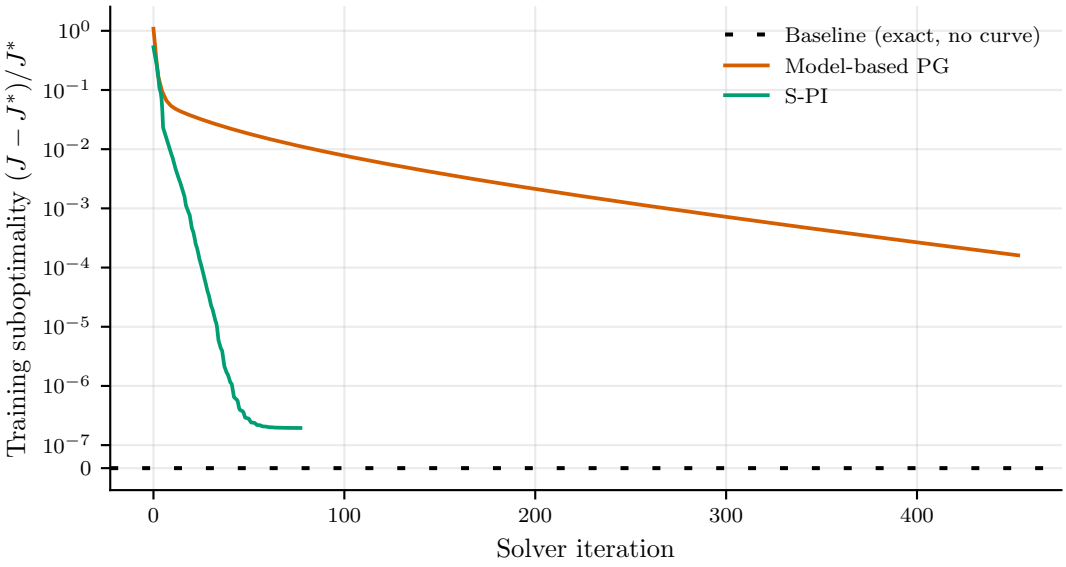


Figure 3.1: TODO

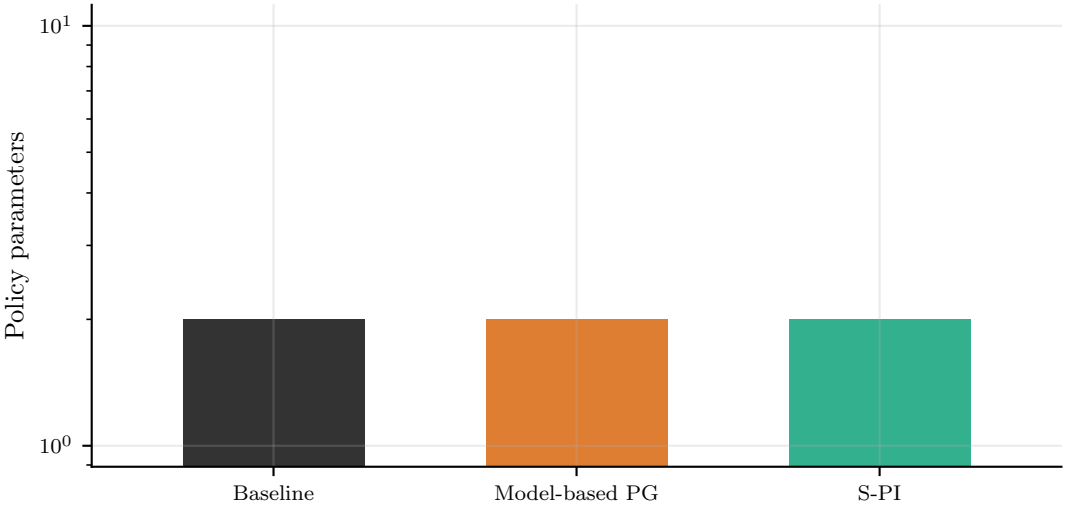


Figure 3.2: TODO

a selection of the results here, and will append further relevant figures in the appendix [CITE: cref]. All plots, including some that were excluded in this thesis, can be found in the code repository [CITE: link to repo].

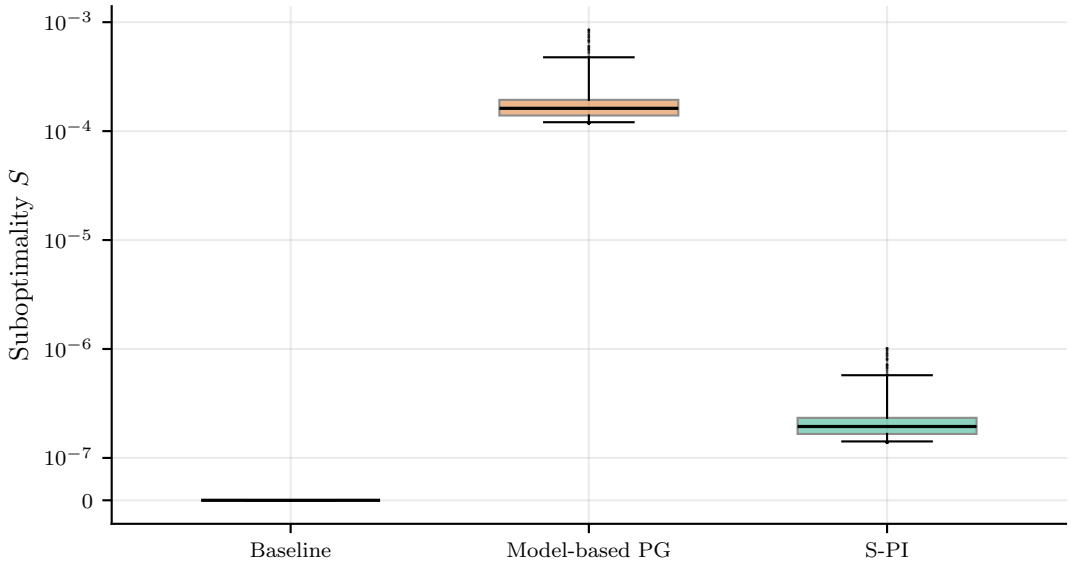


Figure 3.3: TODO

Table 3.1: Policy architectures optimized by the exact gradients on the true model under *base* inflation, without noise. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

Solver	Baseline	Model-based PG	S-PI
True cost J	$5848 \pm 9.587 \times 10^{-13}$	$5849 \pm 2.66 \times 10^{-12}$	$5848 \pm 1.05 \times 10^{-12}$
Training suboptimality $(J - J^*)/J^*$	0 ± 0	$0.000163 \pm 4.3 \times 10^{-16}$	$1.95 \times 10^{-7} \pm 1.23 \times 10^{-16}$
Suboptimality $S^\dagger$	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+2.38 \times 10^{-7} \pm 1.49 \times 10^{-7}$
Iterations	1 ± 0	454 ± 0	78 ± 0
Env. steps to convergence	0 ± 0	0 ± 0	0 ± 0
Env. steps to best policy	0 ± 0	0 ± 0	0 ± 0
Policy parameters	2 ± 0	2 ± 0	2 ± 0
Gain rank	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0

Table 3.2: Policy architectures optimized by the exact gradients on the true model under *large* inflation, without noise. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

Solver	Baseline	Model-based PG	S-PI
True cost J	$5848 \pm 9.587 \times 10^{-13}$	$5849 \pm 2.444 \times 10^{-12}$	$5848 \pm 4.781 \times 10^{-6}$
Training suboptimality $(J - J^*)/J^*$	0 ± 0	$0.000163 \pm 4.18 \times 10^{-16}$	$4.81 \times 10^{-6} \pm 8.18 \times 10^{-10}$
Suboptimality $S^\dagger$	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+5.87 \times 10^{-6} \pm 3.68 \times 10^{-6}$
Iterations	1 ± 0	454 ± 0	77 ± 0
Env. steps to convergence	0 ± 0	0 ± 0	0 ± 0
Env. steps to best policy	0 ± 0	0 ± 0	0 ± 0
Policy parameters	301 ± 0	20000 ± 0	301 ± 0
Gain rank	1.0 ± 0.0	100.0 ± 0.0	1.0 ± 0.0

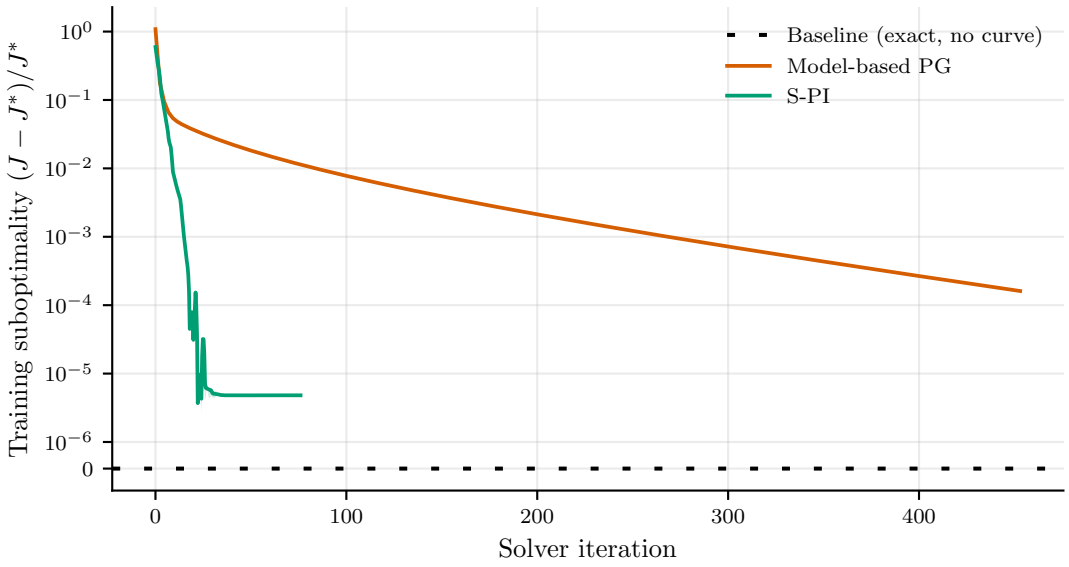


Figure 3.4: TODO

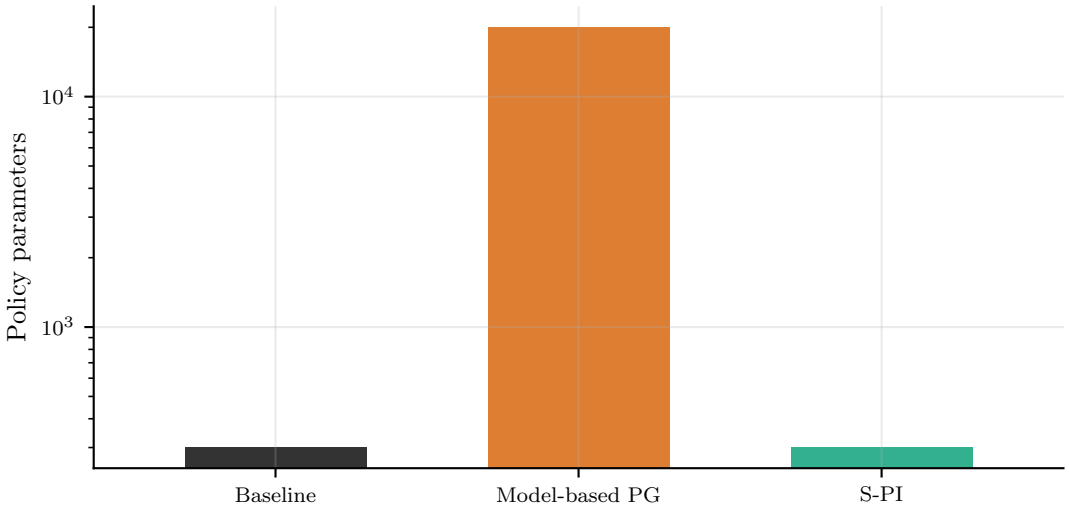


Figure 3.5: TODO

Table 3.3: Policy architectures on the true model, across every inflation size. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

(size, solver)	base, Baseline	base, Model-based PG	base, S-PI	small, Baseline	small, Model-based PG	small, S-PI	medium, Baseline	medium, Model-based PG	medium, S-PI	large, Baseline	large, Model-based PG	large, S-PI
Training suboptimality $(J - J^*)/J^*$	0 ± 0	$0.000163 \pm 4.3 \times 10^{-16}$	$1.95 \times 10^{-7} \pm 1.23 \times 10^{-16}$	0 ± 0	$0.000163 \pm 5.59 \times 10^{-16}$	$4.73 \times 10^{-6} \pm 1.05 \times 10^{-7}$	0 ± 0	$0.000163 \pm 1.83 \times 10^{-16}$	$4.81 \times 10^{-6} \pm 4.49 \times 10^{-8}$	0 ± 0	$0.000163 \pm 4.18 \times 10^{-16}$	$4.81 \times 10^{-6} \pm 8.18 \times 10^{-10}$
Suboptimality S^U	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+2.38 \times 10^{-7} \pm 1.49 \times 10^{-7}$	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+5.76 \times 10^{-6} \pm 3.59 \times 10^{-6}$	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+5.89 \times 10^{-6} \pm 3.69 \times 10^{-8}$	$+0 \pm 0$	$+0.000199 \pm 0.000124$	$+5.87 \times 10^{-6} \pm 3.68 \times 10^{-8}$
Policy parameters	2 ± 0	2 ± 0	2 ± 0	16 ± 0	50 ± 0	16 ± 0	76 ± 0	1250 ± 0	76 ± 0	301 ± 0	20000 ± 0	301 ± 0
Gain rank	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	5.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	25.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	100.0 ± 0.0	1.0 ± 0.0

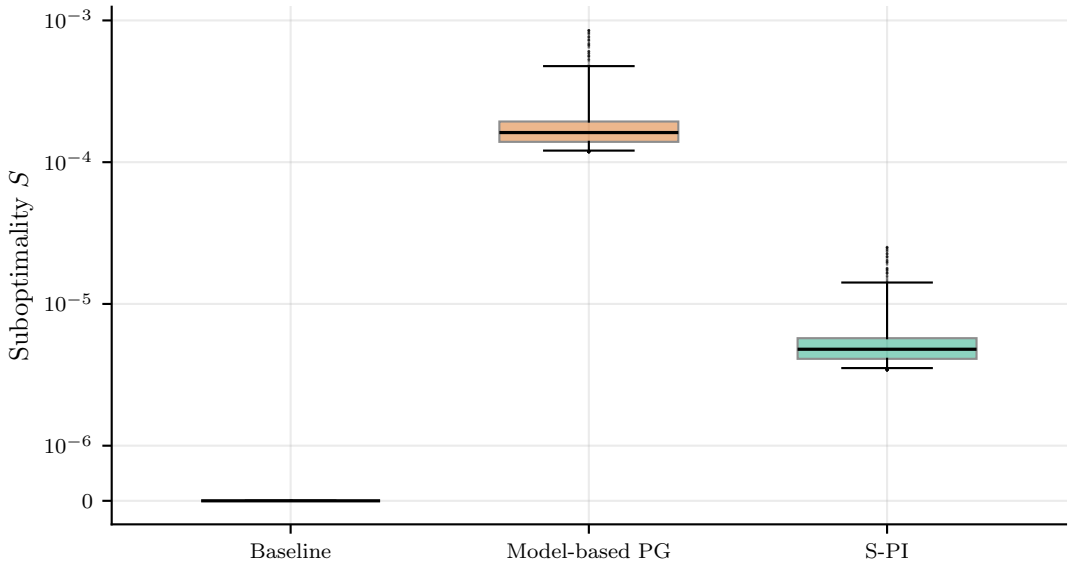


Figure 3.6: TODO

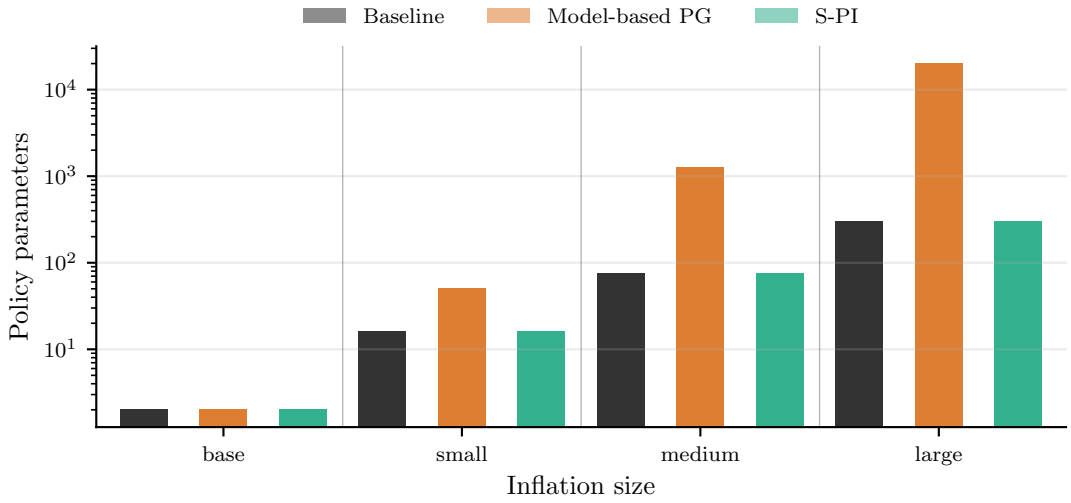


Figure 3.7: TODO

3.1.4 Discussion

3.2 Unknown Model across Dimensions

Here we take a look at the more realistic case, where we do not know the underlying model, and compare different approaches. Before we can get into the results, we first take a deeper look at the solvers that we use and how the experiments are set up.

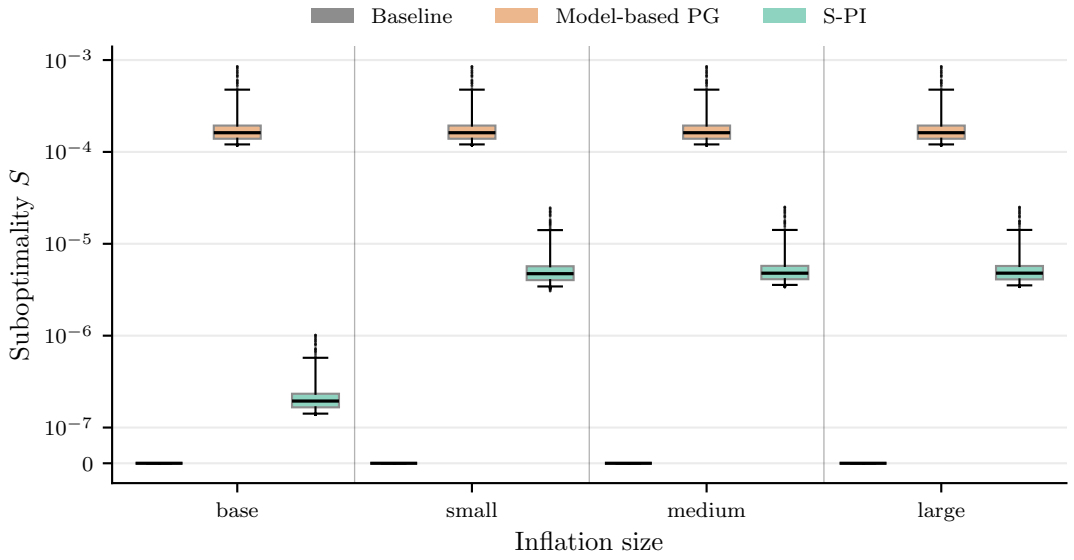


Figure 3.8: TODO

3.2.1 Setting

First, we introduce our basis for the indirect model-free approach, which is based on system identification. N4SID, introduced by Van Overschee and De Moor [38], uses "the projection of the future outputs [of the system] onto the past and future inputs and the past outputs" [38, Sec. 3]. These projections are then called "a function of the system matrices and the input-output block Hankel matrices" [38, Sec. 3], which allows us to reconstruct exactly that from the results. Important are further the assumptions that the systems are observable (Definition 2.4) and controllable (Definition 2.2). For simplicity, we rely on the implementation of N4SID from SIPPY [CITE: TODO], which we integrate into our own setup. This implementation allows us to recover the system matrices A, B and C, whereas we implemented the recovery of the cost matrices Q and R from scratch based on [38].

The collection of observation and action data runs the plant for T steps under the initial gain, with a randomly sampled excitation added on top of it,

$$v_k = -K_0 y_k + \sigma_e w_k, \qquad w_k \sim \mathcal{N}(0, I_{d_{\text{out}}}), \qquad k = 0, \ldots, T-1. \tag{3.5}$$

As mentioned in the paper, "as i grows larger, the approximation error grows smaller" [38, Sec. 5.3], with i being dependent on the length of our trajectory described above used for identification. The other variable j defines the dimensions of the Hankel matrices, and thus how far the projections reach into the future and the past. As we want to be computationally efficient while still achieving the best performance, we evaluated different values for both, to find the optimal setting to use in our experiment. The full results of the selection can be found in the Section C.3. Specifically, we use the settings in Table 3.4.

Table 3.4: Identification settings per inflation size.

Name	Trajectory length T	Hankel horizon
base	500	5
small	500	5
medium	500	5
large	8000	5

The resulting identified system then serves as a surrogate for the true model for the model-based approaches from Section 3.1.1. This thus produces results without ever having access to the real model.

An attentive reader might now ask why we would even want to use these iterative approaches, if we could just solve the DARE (2.20) directly on this estimation. And indeed we include it in our following comparison. However, to make this work with our construction of inflated state models, we need some modification to N4SID. This stems from the fact that we need to calculate $K = \left(R + B^\top P B\right)^{-1} B^\top P A$ from (2.21), hence $R + B^\top P B$ needs to be invertible. But for our inflated space, the estimated B and R have the same null space, making the matrix singular on that subspace. So to be able to apply this solver, we eliminate the additional singular values of B that exceed the estimated order of the model, thus reducing the model in dimension. This way we do not access the model, but create encoding and decoding matrices that map between our dimensionally reduced model and the actual state observation, as well as the actuation output. For the exact implementation, please refer to the repository [CITE: our repository].

Additionally, we also use direct model-free approaches, which are variations of the model-based approaches from Section 3.1. Unlike before, they do not calculate the gradient directly, but estimate it based on the returns of m slightly perturbed versions of the current policy, simulated for l steps. As one might assume, more "data", could conversely to better estimation, and thus better policy updates. So, to settle this, we evaluated different combinations of them in Section C.4. The best trade-off between computational efficiency and quality gave $m = 100$ and $l = 800$.

We use the model-free natural policy gradient (NPG) estimation described by Fazel et al. [11], which at its core consists of the loop in Algorithm 3. Instead of the proposed single-point estimator, we use a two-point estimator inspired by Duchi et al. and Mania et al. [10, 20], as this drastically reduced the variance of the estimates, and thus the instability of the controller.

We further aim to clarify a couple of things about the given loop. Subtracting the mean difference $\bar{\delta}$ does not change the expected value of the estimate, since $\mathbb{E}[U_j] = 0$, but lowers its variance. A rollout whose observation exceeds $\|y_k\| = 10^5$, we count as diverged. It is therefore ended early and charged a penalty cost of

Algorithm 3 Two-point gradient estimation

Require: gain $K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, number of perturbation pairs m, rollout length l, smoothing radius r

for $j = 1, \ldots, m$ **do**
  sample U_j uniformly from the matrices with $\|U_j\|_F = r$
  sample an initial state $x_0 \sim \mathcal{D}$
  simulate $K + U_j$ and $K - U_j$ for l steps from x_0, with stage costs $c_k^{\pm}$ and observations $y_k^{\pm}$
  $\hat{C}_j^{\pm} = \sum_{k=0}^{l-1} c_k^{\pm}, \quad \hat{\Sigma}_j^{\pm} = \sum_{k=0}^{l-1} y_k^{\pm} \left(y_k^{\pm}\right)^{\top}, \quad \delta_j = \hat{C}_j^{+} - \hat{C}_j^{-}$
end for
$\bar{\delta} = \frac{1}{m} \sum_{j=1}^{m} \delta_j$
$\widehat{\nabla f}(K) = \frac{1}{m} \sum_{j=1}^{m} \frac{d_{\text{out}} d_{\text{in}}}{2r^2} \left(\delta_j - \bar{\delta}\right) U_j$
$\hat{\Sigma}_K = \frac{1}{m} \sum_{j=1}^{m} \left(\hat{\Sigma}_j^{+} + \hat{\Sigma}_j^{-}\right)$
$\hat{f}(K) = \frac{1}{m} \sum_{j=1}^{m} \frac{1}{2} \left(\hat{C}_j^{+} + \hat{C}_j^{-}\right)$
return $\widehat{\nabla f}(K)$, $\hat{\Sigma}_K$, $\hat{f}(K)$

10^{10}. Additionally, for inflated dimensions, $\hat{\Sigma}_K$ is singular, so we precondition with its truncated pseudoinverse $\hat{\Sigma}_K^{\dagger}$ instead, which discards singular values below 10^{-6} times the largest one.

Our full learning algorithm is then given in Algorithm 4.

Algorithm 4 Model-free NPG

Require: stabilizing gain $K_0 \in \mathcal{K}$, m, l, r, step size η

for $i = 0, 1, 2, \ldots$ **do**
  compute $\widehat{\nabla f}(K_i)$ and $\hat{\Sigma}_{K_i}$ by Algorithm 3
  update the gain by $K_{i+1} = K_i - \eta \, \widehat{\nabla f}(K_i) \, \hat{\Sigma}_{K_i}^{\dagger}$
end for

After some evaluations, we have decided on a smoothing parameter of $r = 0.5$, and a learning rate of $\eta = 0.01$, based on the results in Section C.5.

Next, we implement a model-free version of S-PI that relies on the same gradient estimation (Algorithm 3) as Model-free NPG. Another crucial difference, apart from the estimation of the gradient compared to the calculation, is the "usage of a fixed stepsize η, rather than an adaptive stepsize from a backtracking linesearch" [25, Sec. 3.1], as that would require access to the function value $f(K) = \operatorname{tr}(\Sigma_0 P_K)$. Before we look at how we select this step size and λ, we define our learning loop in Algorithm 5.

Now to the selection of the parameters. Park et al. already note the difficulty of finding a fixed step size that does not produce divergence of the method [25]. Also, the example shown only uses the model-based approach and the regularizer

Algorithm 5 S-PI (model-free)

Require: stabilizing gain $K_0 \in \mathcal{K}$, m, l, r, fixed step size η, weight λ, tolerance ε

1: **for** $i = 0, 1, 2, \ldots$ **do**
2: compute $\widehat{\nabla f}(K_i)$ by Algorithm 3
3: compute K_{i+1} by (3.3), with $\widehat{\nabla f}(K_i)$ in place of $\nabla f(K_i)$
4: **if** $\|K_{i+1} - K_i\|_F \leq \varepsilon$ **then**
5: **stop**
6: **end if**
7: **end for**

to induce sparse structure, not the model-free variant combined with low dimensionality. So during our first tests, we often ran into either unstable controllers, or ones that did not induce any structure at all, hinting at a too low λ. If we remember the proximal step (3.3), we can see that it depends on both the step size and λ, as every singular value is mapped to

$$\sigma_l \mapsto \max\{\sigma_l - \eta\lambda, 0\}. \tag{3.6}$$

Even after some improvement by using the two-point estimator (Algorithm 3), it remained quite a difficult topic. So rather extensive experiments on the variation of λ and the step size can be found in Section C.6. Based on those, $\lambda = 4000$ and a step size of $\eta = 3.5 \times 10^{-6}$ were selected.

3.2.2 Results

3.2.3 Noise Experiments on Large Inflation

3.2.4 Results

3.2.5 Discussion

3.3 Discussion

Chapter 4

Robustness Of Reinforcement Learning Approaches

4.1 PPO and its Variants

4.2 Comparison Across Dimensions

4.2.1 Setting

4.2.2 Results

4.3 Noise On Large Inflation

4.3.1 Setting

4.3.2 Results

4.4 Discussion

Chapter 5

Perception-based Experiments

5.1 Artificial Inflation to Pixel Size

5.1.1 Setting

5.1.2 Results

5.2 The New Environment and Pos-Vel

5.3 Effect Of Visual Inputs On The Solvers

5.3.1 Setting

5.3.2 Results

5.4 Discussion

Chapter 6

Conclusion

You generally cover three things in the Conclusions section, and each of these usually merits a separate subsection:

1. Conclusions
2. Summary of Contributions
3. Future Research

Conclusions are not a rambling summary of the thesis: they are short, concise statements of the inferences that you have made because of your work. It helps to organize these as short numbered paragraphs, ordered from most to least important. All conclusions should be directly related to the research question stated in the introduction. Examples:

1. The problem stated in the introduction has been solved: as shown in Sections ? to ??, an algorithm capable of handling large-scale Zylon problems in reasonable time has been developed.
2. The principal mechanism needed in the improved Zylon algorithm is the Grooty mechanism.
3. Etc.

The Summary of Contributions will be much sought and carefully read by the examiners. Here you list the contributions of new knowledge that your thesis makes. Of course, the thesis itself must substantiate any claims made here. There is often some overlap with the Conclusions, but that's okay. Concise numbered paragraphs are again best. Organize from most to least important. Examples:

1. Developed a much quicker algorithm for large-scale Zylon problems.
2. Demonstrated the first use of the Grooty mechanism for Zylon calculations.
3. Etc.

The Future Research subsection is included so that researchers picking up this work in future have the benefit of the ideas that you generated while you were working on the project. Again, concise numbered paragraphs are usually best.

Appendix A

Nomenclature

What goes in the appendices? Any material which impedes the smooth development of your presentation, but which is important to justify the results of a thesis. Generally it is material that is of too nitty-gritty a level of detail for inclusion in the main body of the thesis, but which should be available for perusal by the examiners to convince them sufficiently. Examples include program listings, immense tables of data, lengthy mathematical proofs or derivations, etc.

A.1 Notation

The following table contains the conventions regarding symbols used throughout this thesis:

Symbol	**Meaning**
Sets and operators	
I_n, $\mathbf{1}_n$	identity matrix and all-ones vector of dimension n
$\lambda_i(M)$, $\sigma_i(M)$	i-th eigenvalue and i-th singular value of M
$\rho(M)$	spectral radius, $\max_i \lvert\lambda_i(M)\rvert$
$M \succeq 0$, $M \succ 0$	M symmetric positive semi-definite / positive definite
$\lVert\cdot\rVert$, $\lVert\cdot\rVert_F$, $\lVert\cdot\rVert_*$	Euclidean (spectral) norm, Frobenius norm, nuclear norm
$\odot$	elementwise product
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution with mean μ and covariance Σ
∇f, $\widehat{\nabla} f$	gradient of f and the estimate of it
System description	
n, m, q	dimension of the state, input and output
t, k, h	continuous time, discrete step index, sampling period
$x_k \in \mathbb{R}^n$	state

Symbol	**Meaning**
$u_k \in \mathbb{R}^m$	control input computed by the controller
$\tilde{u}_k \in \mathbb{R}^m$	control input actually applied to the plant
$y_k \in \mathbb{R}^q$	observed output
p, v	position and velocity of the double integrator, $x = [\, p,\ v\,]^\top$
A_c, B_c	continuous-time system and input matrix
A, B	their zero-order-hold discretizations
C, D	output and feedthrough matrix; $D = 0$ throughout
A_K	closed-loop matrix, $A_K = A - BK$
$\mathcal{C}(A, B)$, $\mathcal{O}(C, A)$	controllability and observability matrix
$\hat{x}_k$, $\tilde{x}_k$	state estimate and estimation error $x_k - \hat{x}_k$
L	observer gain
Optimal control	
Q, R	state and input cost matrices, $Q \succeq 0$, $R \succ 0$
$K \in \mathbb{R}^{m \times n}$	feedback gain, $u_k = -Kx_k$
K^*	optimal gain, solution of (2.21)
$\mathcal{K}$	set of stabilizing gains, (2.24)
$J(x_0, u)$	infinite-horizon cost of an input sequence
P	stabilizing solution of the DARE (2.20)
P_K	solution of the Lyapunov equation (2.23) for a fixed K
$\mathcal{D}$, Σ_0	distribution of x_0 and its second moment $\mathbb{E}[x_0 x_0^\top]$
$f(K)$	expected cost of a gain, $\mathrm{tr}(P_K \Sigma_0)$
Noise	
n_k^{m}, σ_{m}	measurement noise and its standard deviation
n_k^{c}, σ_{c}	control noise (relative) and its standard deviation
Δ_k^A, Δ_k^B, σ_{p}	parametric perturbations and their standard deviation
n_k^{d}, σ_{d}	external disturbance and its standard deviation / amplitude
Learning and inflated systems	
d_{in}, d_{out}	inflated input and output dimension, $d \gg n$
U_1, U_2	orthonormal inflation matrices
λ	weight of the nuclear-norm regulariser
η	step size
r	smoothing radius of the zeroth-order gradient estimator
N, T	number of rollouts and rollout horizon

A.2 Noise intensities

The settings used for the different noise intensities, obtained by the calibration (2.38) on the non-inflated system. It is important to note that parametric noise is resampled at every step, and that the disturbance noise uses its sinusoidal variant (2.31b).

Table A.2: Standard deviation per noise channel and intensity, calibrated to the cost ratio ρ_ℓ of (2.38). Every value reaches its ratio within 0.9 %.

Channel	*none*	*weak*	*standard*	*strong*	*divergent*
Cost ratio ρ_ℓ	1	1.2	1.5	3	10
Measurement σ_m	0	0.389	0.607	1.22	2.58
Control σ_c	0	1.07	1.67	3.19	5.68
Parametric σ_p	0	0.0211	0.0304	0.0451	0.0582
Disturbance σ_d	0	0.00767	0.0122	0.0246	0.0519

Appendix B

Additional Proofs

Appendix C

Complete Simulation Results

Here the results from some hyperparameter tuning are shown. For simplicity, mostly the ones under *large* inflation are shown, unless named otherwise, as this is the most challenging setting from what we have observed. The remaining plots and tables can be found in the repository[CITE: TODO repo link].

C.1 Learning Rate of Model-based PG

Figure C.1 and Table C.1 show the results for *large* inflation.

Table C.1: Model-based PG under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

Learning rate	1e-06	3e-06	1e-05	3e-05	0.0001	0.0012
True cost J	$5858 \pm 1.768 \times 10^{-12}$	$5852 \pm 1.454 \times 10^{-12}$	$5849 \pm 2.444 \times 10^{-12}$	$5848 \pm 1.356 \times 10^{-12}$	$5848 \pm 7.512 \times 10^{-12}$	5849 ± 0.2438
Training suboptimality $(J - J^\star)/J^\star$	$0.00173 \pm 2.29 \times 10^{-16}$	$0.000606 \pm 2.21 \times 10^{-16}$	$0.000163 \pm 4.18 \times 10^{-16}$	$3.99 \times 10^{-5} \pm 2.1 \times 10^{-16}$	$1.3 \times 10^{-6} \pm 1.27 \times 10^{-15}$	$0.00011 \pm 4.17 \times 10^{-5}$
Test cost†	5978 ± 5281	5972 ± 5276	5969 ± 5273	5968 ± 5273	5968 ± 5272	5969 ± 5273
Iterations	2389 ± 0	1122 ± 0	454 ± 0	1299 ± 0	5449 ± 0	8331 ± 10043
Iterations to 1 %	1038 ± 0	342 ± 0	84 ± 0	1033 ± 0	5330 ± 0	–
Env. steps to 1 %	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0	–
Env. steps to convergence	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0

C.2 Nuclear-Norm Weight of S-PI

Table C.2 and Fig. C.2 show the results for *large* inflation.

C.3 N4SID Identification Length

Figures C.3 and C.4 show the results for *large* and *medium* inflation over all trajectory lengths and Hankel horizons, Tables C.3 and C.4 the detailed results for a Hankel horizon of five.

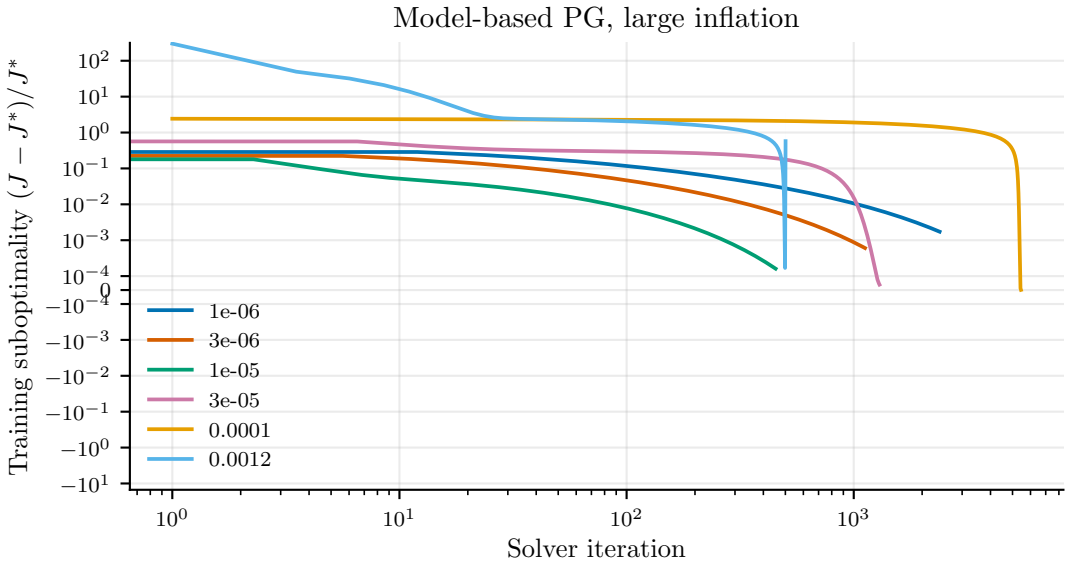


Figure C.1: Training suboptimality of Model-based PG over the solver iterations for each learning rate under *large* inflation. Median and range over ten seeds.

Table C.2: S-PI under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

λ	1	5	20	50	200	1000
True cost J	5848 ± 0.0005895	$5848 \pm 4.781 \times 10^{-6}$	5849 ± 0.002667	5851 ± 0.001226	5878 ± 0.001099	6146 ± 0.000302
Training suboptimality $(J - J^*)/J^*$	$2.6 \times 10^{-7} \pm 1.01 \times 10^{-7}$	$4.81 \times 10^{-6} \pm 8.18 \times 10^{-10}$	$7.38 \times 10^{-5} \pm 4.56 \times 10^{-7}$	$0.000429 \pm 2.1 \times 10^{-7}$	$0.00506 \pm 1.88 \times 10^{-7}$	$0.0509 \pm 5.16 \times 10^{-8}$
Gain rank	4.2 ± 0.8	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
Test cost†	5968 ± 5272	5968 ± 5272	5969 ± 5273	5971 ± 5275	5998 ± 5299	6274 ± 5539
Iterations	155 ± 1	77 ± 0	68 ± 0	72 ± 0	68 ± 1	64 ± 0
Spectral radius	0.9914 ± 0.0000	0.9914 ± 0.0000	0.9916 ± 0.0000	0.9919 ± 0.0000	0.9929 ± 0.0000	0.9952 ± 0.0000

Table C.3: N4SID + Model-based PG on a system with *medium* inflation with a Hankel horizon of 5. Mean ± standard deviation over 10 runs.

Trajectory length T	500	1000	2000	4000	8000
Training suboptimality $(J - J^*)/J^*$	$6.54 \times 10^{-9} \pm 3.78 \times 10^{-9}$	$4.08 \times 10^{-9} \pm 7.25 \times 10^{-10}$	$6.34 \times 10^{-9} \pm 3.65 \times 10^{-9}$	$8.09 \times 10^{-9} \pm 4.78 \times 10^{-9}$	$5.71 \times 10^{-9} \pm 5.91 \times 10^{-9}$
Iterations	258 ± 0	258 ± 0	258 ± 0	258 ± 0	258 ± 0
Env. steps to convergence	500 ± 0	$10^3 \pm 0$	$2 \times 10^3 \pm 0$	$4 \times 10^3 \pm 0$	$8 \times 10^3 \pm 0$
Env. steps to best policy	500 ± 0	$10^3 \pm 0$	$2 \times 10^3 \pm 0$	$4 \times 10^3 \pm 0$	$8 \times 10^3 \pm 0$
Wall time (s)	0.3 ± 0.0	0.5 ± 0.0	0.7 ± 0.0	1.2 ± 0.1	2.4 ± 0.4

C.4 Sample Size

Figures C.5 and C.6 show the results of Model-free NPG for *large* inflation, Table C.5 the training suboptimality for all inflation sizes.

C.5 Learning Rate of Model-free NPG

Figure C.7 and Table C.6 show the results for *large* inflation.

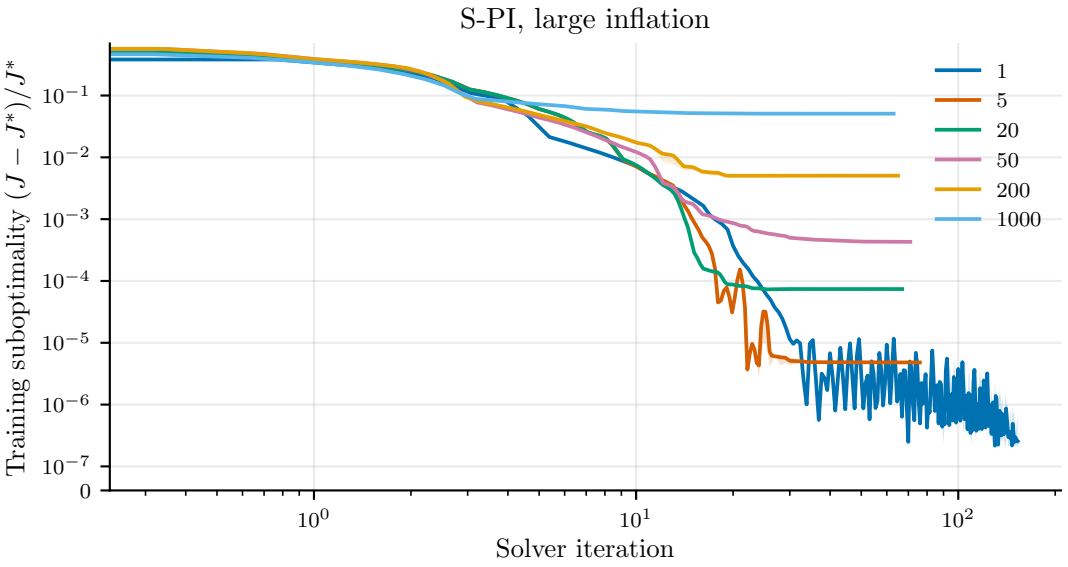


Figure C.2: Training suboptimality of S-PI over the solver iterations for each nuclear-norm weight λ under *large* inflation. Median and range over ten seeds.

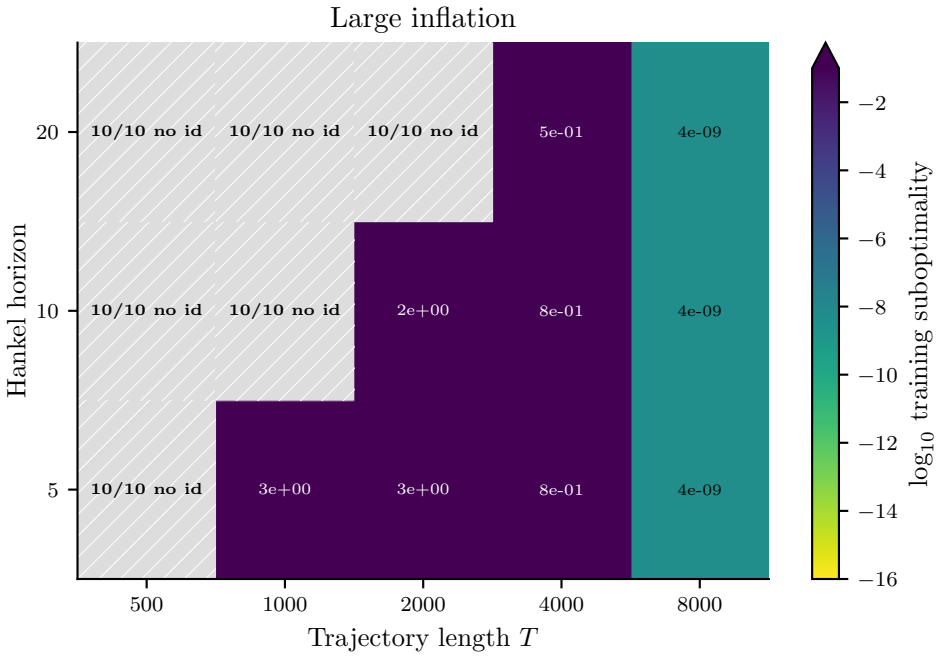


Figure C.3: Median training suboptimality of N4SID + Model-based PG per trajectory length T and Hankel horizon under *large* inflation.

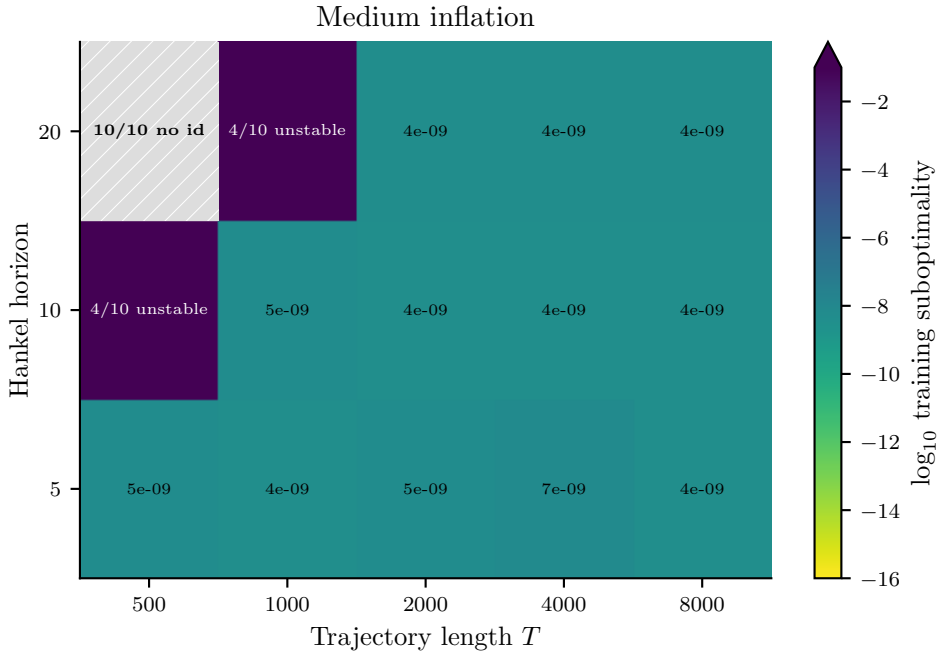


Figure C.4: Median training suboptimality of N4SID + Model-based PG per trajectory length T and Hankel horizon under *medium* inflation.

Table C.4: N4SID + Model-based PG on a system with *large* inflation with a Hankel horizon of 5. Mean ± standard deviation over 10 runs. Identified counts the seeds a model was obtained from and stable those whose controller left the plant stable.

Trajectory length T	500	1000	2000	4000	8000
Identified	0/10	10/10	10/10	10/10	10/10
Stable	0/10	10/10	10/10	10/10	10/10
Training suboptimality $(J - J^*)/J^*$	–	3.45 ± 1.24	2.91 ± 0.434	0.853 ± 0.144	$4.36 \times 10^{-9} \pm 7.78 \times 10^{-10}$
Iterations	–	20000 ± 0	20000 ± 0	20000 ± 0	258 ± 0
Env. steps to convergence	–	$10^3 \pm 0$	$2 \times 10^3 \pm 0$	$4 \times 10^3 \pm 0$	$8 \times 10^3 \pm 0$
Env. steps to best policy	–	$10^3 \pm 0$	$2 \times 10^3 \pm 0$	$4 \times 10^3 \pm 0$	$8 \times 10^3 \pm 0$
Wall time (s)	–	23.2 ± 0.6	29.5 ± 0.3	92.9 ± 13.1	169.0 ± 8.7

Table C.5: Model-free NPG: training suboptimality $(J - J^*)/J^*$ per inflation size (rows) and sample-size combination (columns). Mean ± standard deviation over 10 runs.

(l, m)	200, 100	200, 200	500, 100	500, 200	800, 100	800, 200
base	0.3044 ± 0.003332	0.3019 ± 0.002293	0.05739 ± 0.0009988	0.0567 ± 0.0005461	0.02961 ± 0.0006105	0.02921 ± 0.0003871
small	0.4791 ± 0.03249	0.4785 ± 0.01951	0.05591 ± 0.009796	0.05384 ± 0.006083	0.04145 ± 0.005228	0.04014 ± 0.002976
medium	0.531 ± 0.04661	0.5353 ± 0.03407	0.06065 ± 0.01127	0.06178 ± 0.009836	0.04401 ± 0.002311	0.04413 ± 0.001775
large	0.5369 ± 0.0361	0.5358 ± 0.03296	0.06401 ± 0.01522	0.06321 ± 0.01039	0.04567 ± 0.003642	0.04496 ± 0.002317

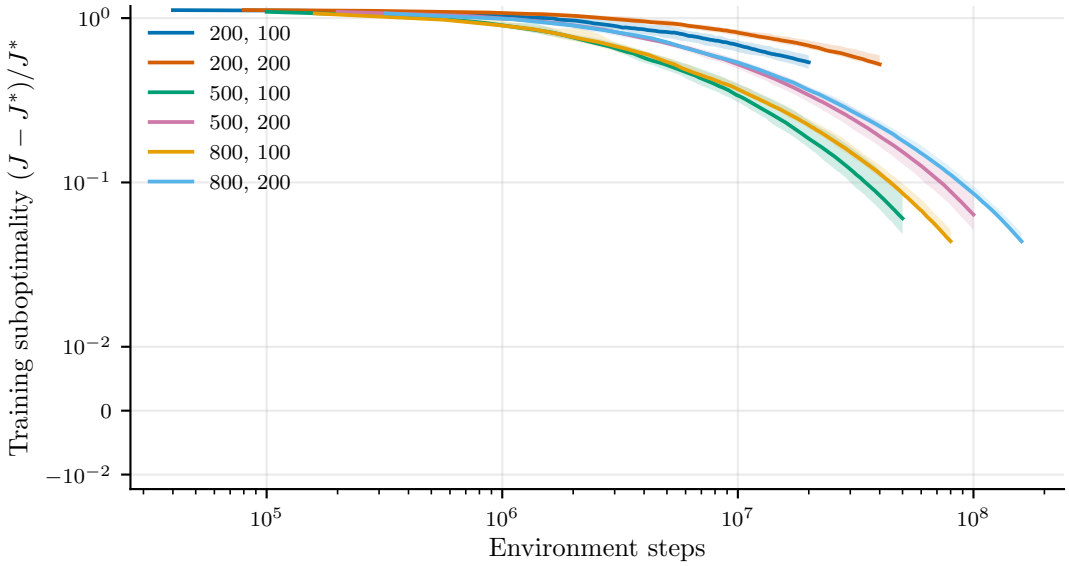


Figure C.5: Training suboptimality of Model-free NPG over the environment steps for each rollout length l and number of trajectories m under *large* inflation. Median and range over ten seeds.

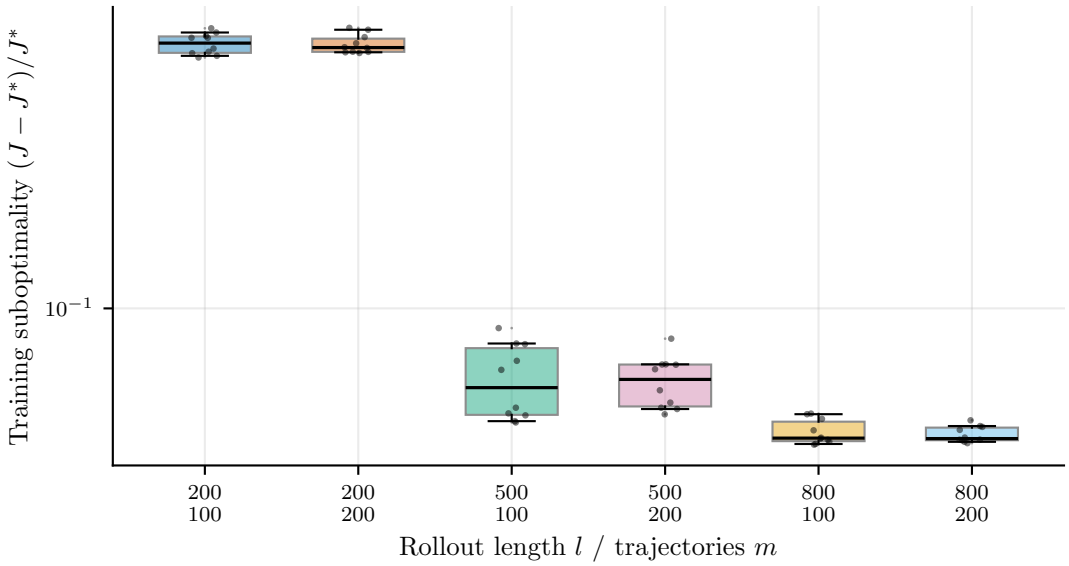


Figure C.6: Final training suboptimality of Model-free NPG for each rollout length l and number of trajectories m under *large* inflation.

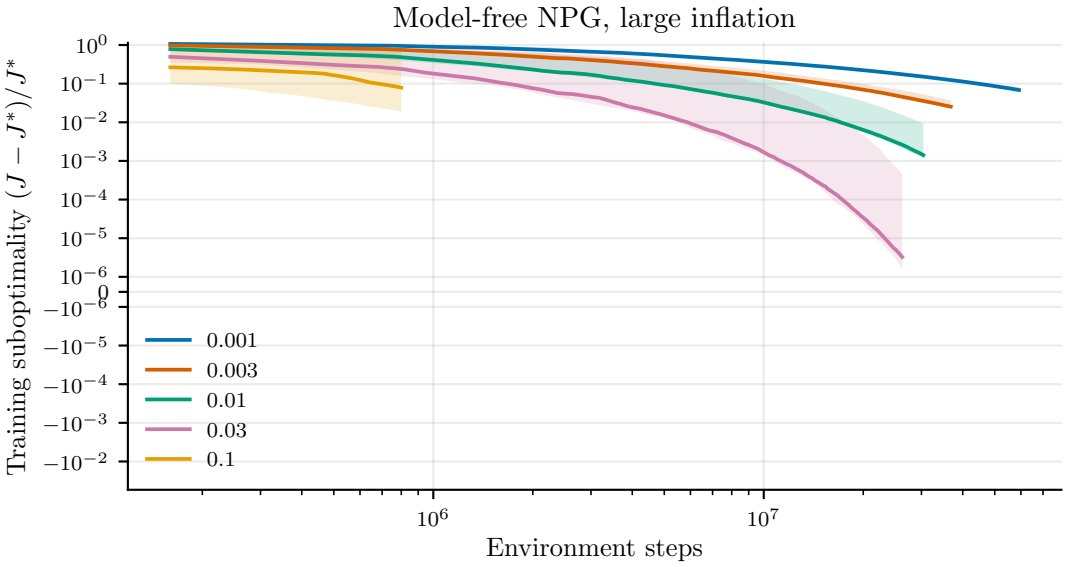


Figure C.7: Training suboptimality of Model-free NPG over the environment steps for each learning rate under *large* inflation. Median and range over ten seeds.

Table C.6: Model-free NPG under *large* inflation, over the seeds. † mean ± standard deviation over all 1000 test episodes (100 per seed, 10 seeds) pooled; every other row over the 10 runs.

Learning rate	0.001	0.003	0.01	0.03	0.1
True cost J	6197 ± 88.89	5960 ± 50.97	5868 ± 11.07	5849 ± 0.5453	6137 ± 913.6
Training suboptimality $(J - J^*)/J^*$	0.0597 ± 0.0152	0.0192 ± 0.00872	0.00341 ± 0.00189	$8.85 \times 10^{-5} \pm 9.32 \times 10^{-5}$	0.0494 ± 0.156
Test cost†	6330 ± 5598	6086 ± 5379	5989 ± 5292	5969 ± 5273	6254 ± 5631
Iterations	475 ± 90	339 ± 57	217 ± 28	194 ± 31	162 ± 26
Iterations to 1 %	–	349 ± 27	116 ± 25	44 ± 23	16 ± 11
Env. steps to 1 %	–	$5.61 \times 10^7 \pm 4.27 \times 10^6$	$1.87 \times 10^7 \pm 4.08 \times 10^6$	$7.14 \times 10^6 \pm 3.69 \times 10^6$	$2.65 \times 10^6 \pm 1.8 \times 10^6$
Env. steps to convergence	$7.6 \times 10^7 \pm 1.44 \times 10^7$	$5.43 \times 10^7 \pm 9.13 \times 10^6$	$3.47 \times 10^7 \pm 4.52 \times 10^6$	$3.11 \times 10^7 \pm 5 \times 10^6$	$2.44 \times 10^7 \pm 8.49 \times 10^6$

C.6 Step Size and Nuclear-Norm Weight of S-PI (Model-Free)

Here, we show different combinations of values for the stepsize and λ under *large* inflation. Excluded are the tables for $\lambda = 400$, as no rank reduction could be observed (λ too low), as well as $\lambda = 4 \times 10^6$, as here all produced controllers were unstable (λ too high).

From the remainders, we picked a value that exhibits rank reduction, while at the same time having a low suboptimality. Tables C.7 to C.9 show the results for the remaining values of λ.

Table C.7: S-PI (model-free) with $\lambda = 4000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs.

Step size	3.5e-09	3.5e-08	3.5e-07	3.5e-06	3.5e-05
True cost J	$1.19 \times 10^4 \pm 398.3$	9139 ± 33.25	$1.059 \times 10^4 \pm 2356$	$1.241 \times 10^4 \pm 4.201 \times 10^{-12}$	$1.241 \times 10^4 \pm 4.537 \times 10^{-12}$
Training suboptimality $(J - J^*)/J^*$	1.03 ± 0.0681	0.563 ± 0.00569	0.811 ± 0.403	$1.12 \pm 6.62 \times 10^{-16}$	$1.12 \pm 6.54 \times 10^{-16}$
Gain rank	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
Spectral radius	0.9975 ± 0.0000	0.9976 ± 0.0000	0.9976 ± 0.0001	0.9975 ± 0.0000	0.9975 ± 0.0000
Iterations	50 ± 0	50 ± 0	50 ± 0	50 ± 0	50 ± 0
Env. steps to convergence	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$
Env. steps to best policy	$4.94 \times 10^6 \pm 3.75 \times 10^6$	$8 \times 10^6 \pm 0$	$3.3 \times 10^6 \pm 4.05 \times 10^6$	$1.6 \times 10^5 \pm 0$	$1.6 \times 10^5 \pm 0$

Table C.8: S-PI (model-free) with $\lambda = 40000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs.

Step size	3.5e-09	3.5e-08	3.5e-07	3.5e-06	3.5e-05
True cost J	$1.162 \times 10^4 \pm 21.42$	9507 ± 32.06	$1.842 \times 10^4 \pm 1398$	$1.241 \times 10^4 \pm 3.688 \times 10^{-12}$	$1.241 \times 10^4 \pm 3.688 \times 10^{-12}$
Training suboptimality $(J - J^*)/J^*$	0.986 ± 0.00366	0.626 ± 0.00548	2.15 ± 0.239	$1.12 \pm 5.87 \times 10^{-16}$	$1.12 \pm 5.87 \times 10^{-16}$
Gain rank	100.0 ± 0.0	76.2 ± 5.3	76.8 ± 5.3	100.0 ± 0.0	100.0 ± 0.0
Spectral radius	0.9975 ± 0.0000	0.9979 ± 0.0000	0.9994 ± 0.0001	0.9975 ± 0.0000	0.9975 ± 0.0000
Iterations	50 ± 0	50 ± 0	50 ± 0	50 ± 0	50 ± 0
Env. steps to convergence	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$7.06 \times 10^6 \pm 2 \times 10^6$
Env. steps to best policy	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$5.92 \times 10^6 \pm 2.5 \times 10^5$	$1.6 \times 10^5 \pm 0$	$1.6 \times 10^5 \pm 0$

Table C.9: S-PI (model-free) with $\lambda = 400000$ under *large* inflation, over the seeds. Mean ± standard deviation over 10 runs. Stable counts the seeds whose controller left the plant stable.

Step size	3.5e-09	3.5e-08	3.5e-07	3.5e-06	3.5e-05
Stable	10/10	10/10	10/10	0/10	0/10
True cost J	$1.208 \times 10^4 \pm 27.22$	$1.255 \times 10^5 \pm 8.548 \times 10^4$	$1.142 \times 10^5 \pm 1.056 \times 10^5$	–	–
Training suboptimality $(J - J^*)/J^*$	1.07 ± 0.00465	20.5 ± 14.6	18.5 ± 18.1	–	–
Gain rank	16.1 ± 0.7	54.6 ± 9.5	19.8 ± 14.6	–	–
Spectral radius	0.9978 ± 0.0000	0.9999 ± 0.0000	0.9998 ± 0.0001	–	–
Iterations	50 ± 0	50 ± 0	50 ± 0	–	–
Env. steps to convergence	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	$8 \times 10^6 \pm 0$	–	–
Env. steps to best policy	$8 \times 10^6 \pm 0$	$6.27 \times 10^6 \pm 2.59 \times 10^5$	$8.16 \times 10^5 \pm 5.06 \times 10^4$	–	–

Bibliography

[1] Alekh Agarwal, Sham Kakade, Akshay Krishnamurthy, and Wen Sun. FLAMBE: Structural complexity and representation learning of low rank MDPs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 20095–20107, 2020.

[2] Nihat Ay. Geometric design principles for brains of embodied agents. *KI – Künstliche Intelligenz*, 29(4):389–399, 2015.

[3] Nicolò Botteghi, Khaled Alaa, Mannes Poel, Beril Sirmacek, Christoph Brune, Abeje Mersha, and Stefano Stramigioli. Low dimensional state representation learning with robotics priors in continuous action spaces. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[4] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[5] Adwait Datar and Nihat Ay. Universal and cheap control architectures for embodied agents. Poster, Institute for Data Science Foundations, Hamburg University of Technology, 2025.

[6] Andrea d'Avella, Philippe Saltiel, and Emilio Bizzi. Combinations of muscle synergies in the construction of a natural motor behavior. *Nature Neuroscience*, 6(3):300–308, 2003.

[7] Sarah Dean, Horia Mania, Nikolai Matni, Benjamin Recht, and Stephen Tu. On the sample complexity of the linear quadratic regulator. *Foundations of Computational Mathematics*, 20(4):633–679, 2020.

[8] Sarah Dean, Nikolai Matni, Benjamin Recht, and Vickie Ye. Robust guarantees for perception-based control. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control (L4DC)*, volume 120 of *PMLR*, 2020. arXiv:1907.03680.

[9] Misha Denil, Babak Shakibi, Laurent Dinh, Marc'Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013. arXiv:1306.0543.

[10] John C. Duchi, Michael I. Jordan, Martin J. Wainwright, and Andre Wibisono. Optimal rates for zero-order convex optimization: The power of two

function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015.

[11] Maryam Fazel, Rong Ge, Sham M. Kakade, and Mehran Mesbahi. Global convergence of policy gradient methods for the linear quadratic regulator. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 1467–1476, 2018.

[12] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[13] Maximilian Igl, Kamil Ciosek, Yingzhen Li, Sebastian Tschiatschek, Cheng Zhang, Sam Devlin, and Katja Hofmann. Generalization in reinforcement learning with selective noise injection and information bottleneck. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1910.12911.

[14] Rico Jonschkowski and Oliver Brock. Learning state representations with robotic priors. *Autonomous Robots*, 39(3):407–428, 2015.

[15] Rico Jonschkowski, Roland Hafner, Jonathan Scholz, and Martin Riedmiller. PVEs: Position-velocity encoders for unsupervised learning of structured state representations. arXiv:1705.09805 [cs.RO], 2017.

[16] Hiroaki Kitano. Biological robustness. *Nature Reviews Genetics*, 5(11):826–837, 2004.

[17] Timothée Lesort, Natalia Díaz-Rodríguez, Jean-François Goudou, and David Filliat. State representation learning for control: An overview. *Neural Networks*, 108:379–392, 2018.

[18] Chenliang Li, Junyu Leng, Jiaxiang Li, Youbang Sun, Shixiang Chen, Shahin Shahrampour, and Alfredo Garcia. AdaRL: Adaptive low-rank structures for robust policy learning under uncertainty. arXiv:2510.11899 [cs.LG], 2025.

[19] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *6th International Conference on Learning Representations (ICLR)*, 2018.

[20] Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search provides a competitive approach to reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018. arXiv:1803.07055.

[21] Tad McGeer. Passive dynamic walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990.

[22] Guido Montúfar, Keyan Ghazi-Zahedi, and Nihat Ay. Geometry and determinism of optimal stationary control in partially observable markov decision processes. *arXiv:1503.07206 [math.OC]*, 2015.

[23] Guido Montúfar, Keyan Ghazi-Zahedi, and Nihat Ay. A theory of cheap control in embodied systems. *PLOS Computational Biology*, 11(5):e1004427, 2015.

[24] Vincent C. Müller and Matej Hoffmann. What is morphological computation? on how the body contributes to cognition and control. *Artificial Life*, 23(1):1–24, 2017.

[25] Youngsuk Park, Ryan A. Rossi, Zheng Wen, Gang Wu, and Handong Zhao. Structured policy iteration for linear quadratic regulator. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. arXiv:2007.06202.

[26] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018.

[27] Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, and Sham Kakade. Towards generalization and simplicity in continuous control. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 6550–6561, 2017.

[28] Venkatesh G. Rao and Dennis S. Bernstein. Naive control of the double integrator. *IEEE Control Systems Magazine*, 21(5):86–97, 2001.

[29] Benjamin Recht. A tour of reinforcement learning: The view from continuous control. *Annual Review of Control, Robotics, and Autonomous Systems*, 2:253–279, 2019.

[30] Benjamin Recht, Maryam Fazel, and Pablo A. Parrilo. Guaranteed minimum-rank solutions of linear matrix equations via nuclear norm minimization. *SIAM Review*, 52(3):471–501, 2010.

[31] Nicholas Roy, Ingmar Posner, Tim Barfoot, Philippe Beaudoin, Yoshua Bengio, Jeannette Bohg, Oliver Brock, Isabelle Depatie, Dieter Fox, Dan Koditschek, Tomas Lozano-Perez, Vikash Mansinghka, Christopher Pal, Blake Richards, Dorsa Sadigh, Stefan Schaal, Gaurav Sukhatme, Denis Thérien, Marc Toussaint, and Michiel Van de Panne. From machine learning to robotics: Challenges and opportunities for embodied intelligence. arXiv:2110.15245 [cs.RO], 2021.

[32] Sergio Rozada and Antonio G. Marques. Matrix low-rank trust region policy optimization. arXiv:2405.17625 [cs.LG], 2024.

[33] Stefan Stojanovic, Yassir Jedra, and Alexandre Proutiere. Model-free low-rank reinforcement learning via leveraged entry-wise matrix estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[34] Shokichi Takakura and Kazuhiro Sato. Structured output feedback control for linear quadratic regulator using policy gradient method. arXiv:2203.00289 [math.OC], 2023.

[35] Shahriar Talebi, Yang Zheng, Spencer Kraisler, Na Li, and Mehran Mesbahi. Policy optimization in control: Geometry and algorithmic implications. arXiv:2406.04243 [math.OC], 2024.

[36] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

[37] Aleksandar Todorov and Matthia Sabatelli. Learning in low-dimensional subspaces: Orthogonal bottlenecks for reinforcement learning. *Reinforcement Learning Journal*, 2026. arXiv:2605.26012.

[38] Peter Van Overschee and Bart De Moor. N4SID: Subspace algorithms for the identification of combined deterministic-stochastic systems. *Automatica*, 30(1):75–93, 1994.

[39] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. arXiv:1506.07365.

[40] Jingxi Xu, Bruce Lee, Nikolai Matni, and Dinesh Jayaraman. How are learned perception-based controllers impacted by the limits of robust control? In *Proceedings of the 3rd Conference on Learning for Dynamics and Control (L4DC)*, volume 144 of *PMLR*, 2021. arXiv:2104.00827.

[41] Keyan Zahedi and Nihat Ay. Quantifying morphological computation. *Entropy*, 15(5):1887–1915, 2013.

[42] Kaiqing Zhang, Bin Hu, and Tamer Başar. Policy optimization for $\mathcal{H}_2$ linear control with $\mathcal{H}_\infty$ robustness guarantee: Implicit regularization and global convergence. *SIAM Journal on Control and Optimization*, 59(6):4081–4109, 2021.

[43] Feiran Zhao, Florian Dörfler, Alessandro Chiuso, and Keyou You. Data-enabled policy optimization for direct adaptive learning of the LQR. arXiv:2401.14871 [math.OC], 2024.

Erklärung der Urheberschaft

Ich versichere an Eides statt, dass ich die Bachelor's Thesis im Studiengang Data Science selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen – benutzt habe. Es wurden keine Quellen genutzt, die nicht genannt wurden. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe und die eingereichte schriftliche Fassung der auf dem elektronischen Speichermedium entspricht.

Ort, Datum Unterschrift

Erklärung zur Veröffentlichung

Ich erkläre mein Einverständnis mit der Einstellung dieser Bachelor's Thesis in den Bestand der Bibliothek.

Ort, Datum Unterschrift